Team Members

Name: Georgios Sidiropoulos AM: 4789

Name: Pantelis Bonitsis AM: 4742

---

1. Initialization

Essential libraries

```
# Standard Libraries
import os
import time
import json
import gc
import warnings
from itertools import product

# Data Handling & Visualization
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.notebook import tqdm
from IPython.display import FileLink

# Image Processing
import cv2

# Ignore Warnings
warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore')

# TensorFlow/Keras
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, GlobalAveragePooling2D,
    Flatten, Dense, Dropout, BatchNormalization
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, Callback
from tensorflow.keras.backend import clear_session
```

```
# Pretrained CNNs
from tensorflow.keras.applications import (
    ResNet50, VGG16, EfficientNetB0,
    MobileNetV2, InceptionV3
)
from tensorflow.keras.applications.resnet50 import preprocess_input as
resnet_preprocess
from tensorflow.keras.applications.vgg16 import preprocess_input as
vgg_preprocess
from tensorflow.keras.applications.efficientnet import
preprocess_input as efficientnet_preprocess
from tensorflow.keras.applications.mobilenet_v2 import
preprocess_input as mobilenet_preprocess
from tensorflow.keras.applications.inception_v3 import
preprocess_input as inception_preprocess

# Sklearn Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import (
    RandomForestClassifier, BaggingClassifier, AdaBoostClassifier
)
from sklearn.tree import DecisionTreeClassifier

# Sklearn Evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score, roc_curve
)
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize
```

```
The history saving thread hit an unexpected error
(OperationalError('attempt to write a readonly database')).History
will not be written to the database.
```

```
2025-05-08 10:57:34.239994: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1746701854.443484      31 cuda_dnn.cc:8310] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1746701854.501367      31 cuda_blas.cc:1418] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
```

Dataset Paths

```
# Root dataset directory
dataset_path = '/kaggle/input/machine-learning-undergraduate-course-data/machine-learning-undergraduate-course-cse-uoi-gr'

# Training image directories
pleasant_images_path = os.path.join(dataset_path, 'train', 'pleasant')
unpleasant_images_path = os.path.join(dataset_path, 'train', 'unpleasant')

# Test images directory and ID CSV
test_images_dir = os.path.join(dataset_path, 'TEST_images')
test_csv_path = os.path.join(dataset_path, 'Test-IDs.csv')
```

Data Preparation: Loading, Preprocessing, and Resizing of Images

Data Loading and Initial Exploration

In this section, we:

- Define the paths to the dataset folders containing pleasant and unpleasant images.
- Implement a **function to load images**:
  - Read each image from the folder.
  - Record the original size (height and width) of each image.
  - Assign the appropriate label (`1` for pleasant, `0` for unpleasant).
- **Combine** all loaded images and labels into unified arrays for further processing.

Image Size Analysis

After loading the dataset:

- We create a **DataFrame** to store the original height and width of all images.
- We **analyze the distribution** of image dimensions by computing descriptive statistics (mean, std, min, max, etc.).
- We **visualize** the distribution of image sizes using histograms, separately for height and width.

Class Distribution Analysis

Finally, we:

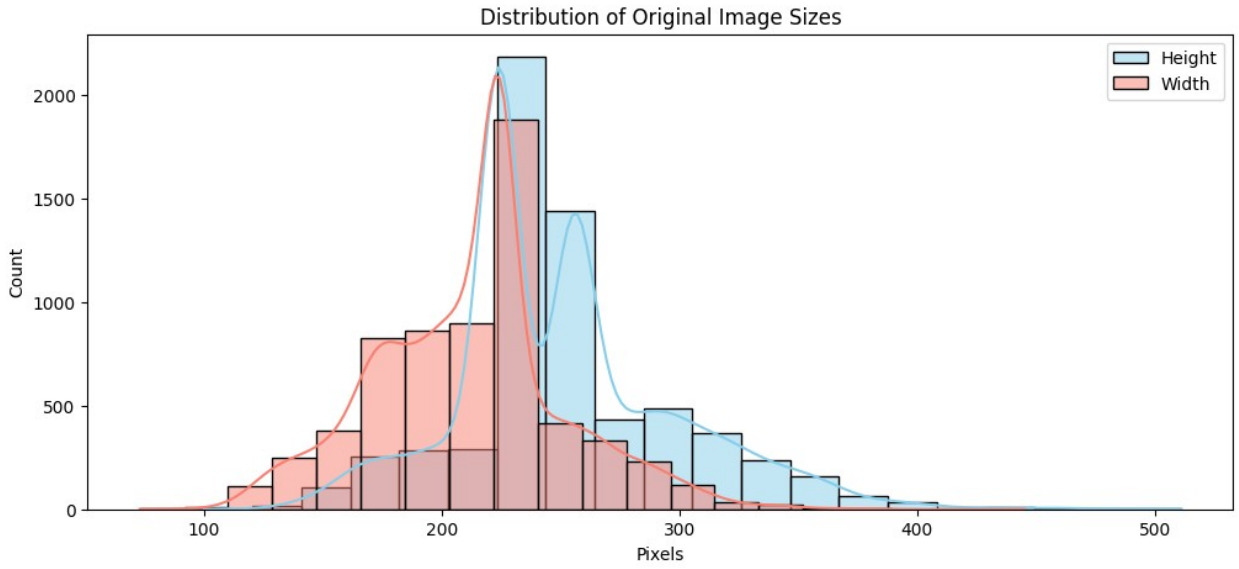
- Count the number of images belonging to each class (pleasant and unpleasant).
- **Visualize** the class distribution using a bar plot.

```
# Function to load images, capture sizes
def load_and_resize_images(folder, label):
```

```
    images = []
    labels = []
    original_sizes = []

    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        img = cv2.imread(img_path)
        if img is not None:
            h, w = img.shape[:2]
            original_sizes.append((h, w))
            images.append(img)
            labels.append(label)

    return images, labels, original_sizes

# Load pleasant images
pleasant_images, pleasant_labels, pleasant_sizes =
load_and_resize_images(pleasant_images_path, 1)

# Load unpleasant images
unpleasant_images, unpleasant_labels, unpleasant_sizes =
load_and_resize_images(unpleasant_images_path, 0)

# Combine data
images = pleasant_images + unpleasant_images
labels = np.array(pleasant_labels + unpleasant_labels)
original_sizes = pleasant_sizes + unpleasant_sizes

##########################################################################
####################
# Original sizes analysis
sizes_df = pd.DataFrame(original_sizes, columns=['Height', 'Width'])
print("\nOriginal Image Sizes:")
print(sizes_df.describe())

# Size distribution visualization
plt.figure(figsize=(12, 5))
sns.histplot(sizes_df['Height'], bins=20, kde=True, color='skyblue',
label='Height')
sns.histplot(sizes_df['Width'], bins=20, kde=True, color='salmon',
label='Width')
plt.xlabel('Pixels')
plt.ylabel('Count')
plt.title('Distribution of Original Image Sizes')
plt.legend()
plt.show()

##########################################################################
####################
# Class distribution visualization
```

```
class_counts = pd.Series(labels).value_counts()
print("\nClass distribution:\n", class_counts)

sns.barplot(x=class_counts.index, y=class_counts.values)
plt.title('Class Distribution (0: Unpleasant, 1: Pleasant)')
plt.xlabel('Class')
plt.ylabel('Number of Images')
plt.show()
```

```
Original Image Sizes:
            Height        Width
count  6376.000000  6376.000000
mean    251.094260   211.977258
std      47.888656    40.171202
min     100.000000    73.000000
25%     224.000000   185.000000
50%     247.000000   218.000000
75%     274.000000   226.000000
max     511.000000   445.000000
```




```
Class distribution:
 1    3394
0    2982
Name: count, dtype: int64
```

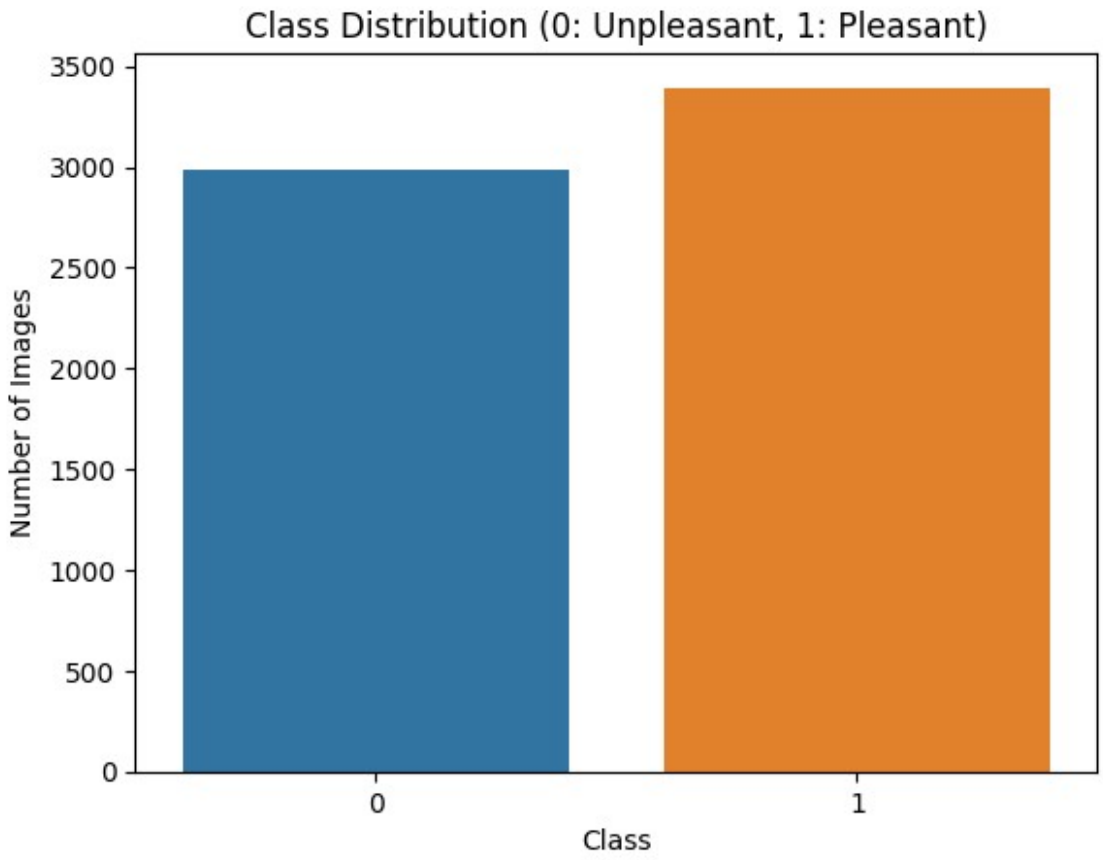


Image Preprocessing and Resizing Strategy

To prepare the images for training and testing, we implemented a function that:

1. Resizes it to a fixed target size of 224 × 224 pixels.
2. Normalizes the pixel values to the range [0, 1].

We selected the size 224×224 for two main reasons:

1. Consistent input size: CNNs require fixed-size inputs. Resizing ensures all images have the same shape, avoiding tensor shape mismatches.
2. Based on dataset statistics: The original dataset had an average size of approximately 251×212 pixels. The value 224 is close to this average.

Interpolation Strategy

We applied different interpolation methods depending on whether an image is being upscaled or downscaled:

1. Downscaling: We use `cv2.INTER_AREA` which performs area-based resampling. This reduces aliasing and retains important image features.

2. Upscaling: We use `cv2.INTER_CUBIC` which uses bicubic interpolation, producing smoother, high-quality enlarged images.

Normalization

After resizing, we normalize pixel values:

$$\text{pixel} = \frac{\text{pixel}}{255.0}$$

This scales all values to $[0, 1)$, which helps the CNN train more efficiently.

```
# Function dynamically select interpolation and resize the images
def resize_images(images, target_dim=(224, 224)):
    resized_images = []  # New list to collect resized images
    scaling_info = {'upscale': 0, 'downscale': 0, 'mixed': 0}

    for img in images:
        h, w = img.shape[:2]

        # Check scaling type
        upscale_h = h < target_dim[0]
        upscale_w = w < target_dim[1]

        # Determine dominant scaling type
        if upscale_h and upscale_w:
            interpolation = cv2.INTER_CUBIC  # both upscale
            scaling_info['upscale'] += 1
        elif not upscale_h and not upscale_w:
            interpolation = cv2.INTER_AREA   # both downscale
            scaling_info['downscale'] += 1
        else:
            # Mixed scaling: choose INTER_AREA due to dominant
vertical downscaling
            interpolation = cv2.INTER_AREA
            scaling_info['mixed'] += 1

        # Resize
        img_resized = cv2.resize(img, target_dim,
interpolation=interpolation)
        resized_images.append(img_resized)

    return resized_images, scaling_info

# Scale images
images, total_scaling_info = resize_images(images)

images = np.array(images)
```

```
print(total_scaling_info)

# Visualize sample images after resize
def plot_sample_images(images, labels, num_samples=5):
    plt.figure(figsize=(15, 5))
    idxs = np.random.choice(len(images), num_samples, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(1, num_samples, i + 1)
        plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
        plt.title(f"Label: {labels[idx]}")
        plt.axis('off')
    plt.show()

print("\nSample Images After Resize:")
plot_sample_images(images, labels)

# Normalize images for CNN
images = images / 255.0
```

```
{'upscale': 939, 'downscale': 2941, 'mixed': 2496}
```

```
Sample Images After Resize:
```

---

2. Direct Image-Based Classification using a CNN hyperparameters tuning

Data Splitting

We begin by splitting the dataset into training and validation sets (80/20 split). We also free memory by deleting the original full dataset variables.

```
# Split dataset (80% train, 20% validation)
X_train, X_val, y_train, y_val = train_test_split(
    images, labels, test_size=0.2, random_state=42, stratify=labels
)
```

```
del images, labels
gc.collect()

print("\nTrain set shape:", X_train.shape, y_train.shape)
print("Validation set shape:", X_val.shape, y_val.shape)
```

```
Train set shape: (5100, 224, 224, 3) (5100,)
Validation set shape: (1276, 224, 224, 3) (1276,)
```

Functions for CNN Training, Hyperparameters Tuning and Evaluation

This section defines the functions used for the CNN Hyperparameters tuning pipeline.

`build_cnn_model()`

This function constructs a CNN architecture with the following configurable parameters:

- `num_conv_layers`: Number of convolutional layers (Conv2D,BatchNorm,MaxPooling)
- `conv_filters`: Base number of filters (doubles per layer, capped at 128)
- `num_dense_layers`: Number of fully connected (dense) layers after flattening
- `dense_units`: Initial size of dense layers (halved at each layer)
- `dropout_rate`: Dropout regularization to reduce overfitting

The model concludes with a softmax output for classification using crossentropy loss.

`evaluate_model()`

Evaluates a trained CNN on validation data and computes:

- Classification metrics: accuracy, precision, recall, F1-score, ROC AUC
- Optional plots:
  - Confusion matrix
  - ROC curve
  - 10 randomly selected misclassified images (with true/predicted labels)

`run_grid_search()`

This function runs a manual grid search across all combinations of selected hyperparameters:

- Builds and trains a model for each configuration
- Evaluates it using `evaluate_model()`
- Records training history and validation metrics
- Saves results to a CSV

It returns a sorted DataFrame of all runs ranked by F1-score.

```
def build_cnn_model(input_shape, num_conv_layers, num_dense_layers,
conv_filters, dense_units, dropout_rate):
    model = Sequential()

    # First Conv layer
    model.add(Conv2D(conv_filters, (3, 3), activation='relu',
input_shape=input_shape))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))

    # Additional Conv layers
    for i in range(1, num_conv_layers):
        filters = min(conv_filters * (2 ** i), 128)  # cap filters
        model.add(Conv2D(filters, (3, 3), activation='relu'))
        model.add(BatchNormalization())
        model.add(MaxPooling2D((2, 2)))

    model.add(Flatten())

    # Dense layers with progressive reduction
    for i in range(num_dense_layers):
        units = dense_units // (2 ** i)
        model.add(Dense(units, activation='relu'))
        model.add(Dropout(dropout_rate))

    # Output layer
    model.add(Dense(2, activation='softmax'))

    model.compile(optimizer=Adam(),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

def evaluate_model(model, X_val, y_val, plots=False):
    y_prob = model.predict(X_val)
    y_pred = np.argmax(y_prob, axis=1)

    acc = accuracy_score(y_val, y_pred)
    prec = precision_score(y_val, y_pred)
    rec = recall_score(y_val, y_pred)
    f1 = f1_score(y_val, y_pred)
    roc_auc = roc_auc_score(y_val, y_prob[:, 1])
    cm = confusion_matrix(y_val, y_pred)

    if plots:
        # Confusion Matrix
        plt.figure(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
        plt.title("Confusion Matrix")
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
```

```
        plt.show()

        # ROC Curve
        fpr, tpr, _ = roc_curve(y_val, y_prob[:, 1])
        plt.figure(figsize=(6, 4))
        plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
        plt.plot([0, 1], [0, 1], '--', color='gray')
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title("ROC Curve")
        plt.legend()
        plt.grid()
        plt.show()

        # Misclassified images
        wrong_indices = np.where(y_pred != y_val)[0]
        if len(wrong_indices) > 0:
            sample_wrong = np.random.choice(wrong_indices,
size=min(10, len(wrong_indices)), replace=False)

            plt.figure(figsize=(15, 6))
            for i, idx in enumerate(sample_wrong):
                plt.subplot(2, 5, i + 1)
                img = X_val[idx]
                if img.ndim == 2 or img.shape[-1] == 1:
                    plt.imshow(img.squeeze(), cmap='gray')
                else:
                    plt.imshow(img)
                plt.title(f"True: {y_val[idx]}\nPred: {y_pred[idx]}")
                plt.axis('off')
            plt.suptitle("Sample Misclassified Images", fontsize=16)
            plt.tight_layout()
            plt.show()
        else:
            print("No misclassified samples to display.")

    print(f" - F1 Score:  {f1:.4f}")
    print(f" - ROC AUC:   {roc_auc:.4f}")
    return {
        'accuracy': acc,
        'precision': prec,
        'recall': rec,
        'f1_score': f1,
        'roc_auc': roc_auc
    }

def run_grid_search(param_combinations, param_names, epochs=10,
results_filename="cnn_gridsearch_results.csv"):
    results = []
```

```
    for combo in tqdm(param_combinations, desc="Grid Search
Progress"):
        config = dict(zip(param_names, combo))
        print(f"\nTraining Config: {config}")

        # Build model
        model = build_cnn_model(
            input_shape=(224, 224, 3),
            num_conv_layers=config['num_conv_layers'],
            num_dense_layers=config['num_dense_layers'],
            conv_filters=config['conv_filters'],
            dense_units=config['dense_units'],
            dropout_rate=config['dropout_rate']
        )

        start = time.time()
        history = model.fit(
            X_train, y_train,
            validation_data=(X_val, y_val),
            epochs=epochs,
            batch_size=config['batch_size'],
            verbose=0
        )
        end = time.time()

        # Evaluate model
        metrics = evaluate_model(model, X_val, y_val)
        metrics['training_time'] = round(end - start, 2)

        # Save training history
        history_json = json.dumps(history.history)

        # Store the full result
        results.append({**config, **metrics, 'history': history_json})

        # Save progress incrementally
        pd.DataFrame(results).to_csv(results_filename, index=False)

        # Cleanup memory
        del model, history, history_json
        gc.collect()

    # Create final sorted DataFrame
    results_df = pd.DataFrame(results).sort_values(by="f1_score",
ascending=False)
    return results_df
```

Define Hyperparameter Search Grids

Multiple hyperparameter grids are defined to explore combinations of CNN configurations. Each grid varies:

- Number of convolutional layers (`num_conv_layers`)
- Number of dense layers (`num_dense_layers`)
- Dropout rate (`dropout_rate`)
- Batch size (`batch_size`)
- Base number of convolution filters (`conv_filters`)
- Width of dense units (`dense_units`)

Grids are grouped by batch size and dense width.

Each grid was executed independently, and results were saved to separate CSV files.
This strategy was necessary because running all configurations in a single session caused GPU memory to exceed its allocation, leading to session crashes in the notebook environment.

To manage this:

- We ran one grid at a time.
- After each grid completed, we saved its results to a csv.
- We then restarted the session to clear GPU memory before running the next grid.

This approach ensured that the system remained stable and all configurations could be evaluated without resource exhaustion.

```
# Grid 1
param_grid_batch_16_128 = {
    'num_conv_layers': [2, 3, 4, 5],          # Required: 2–5 conv
layers
    'num_dense_layers': [1, 2, 3],            # Fully connected layers
    'dropout_rate': [0.3, 0.5],               # Optional
regularization
    'batch_size': [16],                     # Common option
    'conv_filters': [16, 32],                   # Base filter count
    'dense_units': [128]                    # Width of dense layers
}

# Grid 2
param_grid_batch_16_256 = {
    'num_conv_layers': [2, 3, 4, 5],
    'num_dense_layers': [1, 2, 3],
    'dropout_rate': [0.3, 0.5],
    'batch_size': [16],
    'conv_filters': [16, 32],
    'dense_units': [256]
}

# Grid 3
```

```
param_grid_batch_32_128 = {
    'num_conv_layers': [2, 3, 4, 5],
    'num_dense_layers': [1, 2, 3],
    'dropout_rate': [0.3, 0.5],
    'batch_size': [32],
    'conv_filters': [16, 32],
    'dense_units': [128]
}

# Grid 4
param_grid_batch_32_256 = {
    'num_conv_layers': [2, 3, 4, 5],
    'num_dense_layers': [1, 2, 3],
    'dropout_rate': [0.3, 0.5],
    'batch_size': [32],
    'conv_filters': [16, 32],
    'dense_units': [256]
}

param_combinations_batch_16_128 =
list(product(*param_grid_batch_16_128.values()))
param_names_batch_16_128 = list(param_grid_batch_16_128.keys())
print(f"Total configs to test for batch 16:
{len(param_combinations_batch_16_128)}")

results_df_1 = run_grid_search(param_combinations_batch_16_128,
param_names_batch_16_128, epochs=10,
results_filename="cnn_gridsearch_results_batch_size_16_128.csv")

param_combinations_batch_16_256 =
list(product(*param_grid_batch_16_256.values()))
param_names_batch_16_256 = list(param_grid_batch_16_256.keys())
print(f"Total configs to test for batch 16:
{len(param_combinations_batch_16_256)}")

results_df_2 = run_grid_search(param_combinations_batch_16_256,
param_names_batch_16_256, epochs=10,
results_filename="cnn_gridsearch_results_batch_size_16_256.csv")

param_combinations_batch_32_128 =
list(product(*param_grid_batch_32_128.values()))
param_names_batch_32_128 = list(param_grid_batch_32_128.keys())
print(f"Total configs to test for batch 32:
{len(param_combinations_batch_32_128)}")#

results_df_3 = run_grid_search(param_combinations_batch_32_128,
param_names_batch_32_128, epochs=10,
results_filename="cnn_gridsearch_results_batch_size_32_128.csv") #
batch_size 32
```

```
param_combinations_batch_32_256 =
list(product(*param_grid_batch_32_256.values()))
param_names_batch_32_256 = list(param_grid_batch_32_256.keys())
print(f"Total configs to test for batch 32:
{len(param_combinations_batch_32_256)}")

results_df_4 = run_grid_search(param_combinations_batch_32_256,
param_names_batch_32_256, epochs=10,
results_filename="cnn_gridsearch_results_batch_size_32_256.csv") #
batch_size 32
```

Load and Combine Grid Search Results

After completing the grid searches, we load all resulting CSV files from the output directory and concatenate them into a single DataFrame. This unified table allows for comprehensive comparison and analysis of all CNN configurations evaluated across different parameter grids.

The `df_result` DataFrame contains performance metrics and training times for each model.

```
# Directory where the input datasets are mounted in Kaggle
input_dir = '/kaggle/input/cnn-grid-results'

# List all CSV files in the directory
csv_files = [f for f in os.listdir(input_dir) if f.endswith('.csv')]

# Read and concatenate all CSVs into one DataFrame (no 'source'
column)
df_result = pd.concat(
    [pd.read_csv(os.path.join(input_dir, file)) for file in
csv_files],
    ignore_index=True
)

# Show the first few rows
df_result.head()
```

```
   num_conv_layers  num_dense_layers  dropout_rate  batch_size
conv_filters  \
0                2                 1           0.3          32
16
1                2                 1           0.3          32
32
2                2                 1           0.5          32
16
3                2                 1           0.5          32
32
4                2                 2           0.3          32
16

   dense_units  accuracy  precision    recall  f1_score   roc_auc  \
```

```
0           256  0.777429   0.718232  0.957290  0.820707  0.904866
1           256  0.755486   0.719235  0.886598  0.794195  0.839182
2           256  0.829154   0.967546  0.702504  0.813993  0.962947
3           256  0.797806   0.958606  0.648012  0.773286  0.936098
4           256  0.905172   0.934579  0.883652  0.908403  0.972737

   training_time                                              history
0          55.53  {"accuracy": [0.7752941250801086, 0.8323529362...
1          73.61  {"accuracy": [0.7729411721229553, 0.8274509906...
2          51.58  {"accuracy": [0.775882363319397, 0.84117645025...
3          69.69  {"accuracy": [0.7527450919151306, 0.8190196156...
4          54.17  {"accuracy": [0.7111764550209045, 0.7607843279...
```

Results Analysis & Visualization

After aggregating the grid search results, we:

- Display the top 10 models by F1-score
- Compute average F1-scores grouped by each hyperparameter (e.g., convolutional layers, dropout, filters)
- Visualize key trends with:
  - Heatmap of F1-score by combinations of conv and dense layers
  - Boxplots for F1-score vs. number of conv layers and dropout rate

We also output the best configuration discovered across all tested models based on validation F1-score.

```
# Top 10 models
print("Top 10 Models by F1 Score:")
display(df_result.sort_values(by="f1_score",
ascending=False).head(10))

# F1 score grouped by key hyperparameters
print("\n Average F1 Score by Number of Convolutional Layers:")
print(df_result.groupby("num_conv_layers")
["f1_score"].mean().sort_values(ascending=False))

print("\n Average F1 Score by Number of Dense Layers:")
print(df_result.groupby("num_dense_layers")
["f1_score"].mean().sort_values(ascending=False))

print("\n Average F1 Score by Dropout Rate:")
print(df_result.groupby("dropout_rate")
["f1_score"].mean().sort_values(ascending=False))

print("\n Average F1 Score by Convolutional Filters:")
print(df_result.groupby("conv_filters")
["f1_score"].mean().sort_values(ascending=False))
```

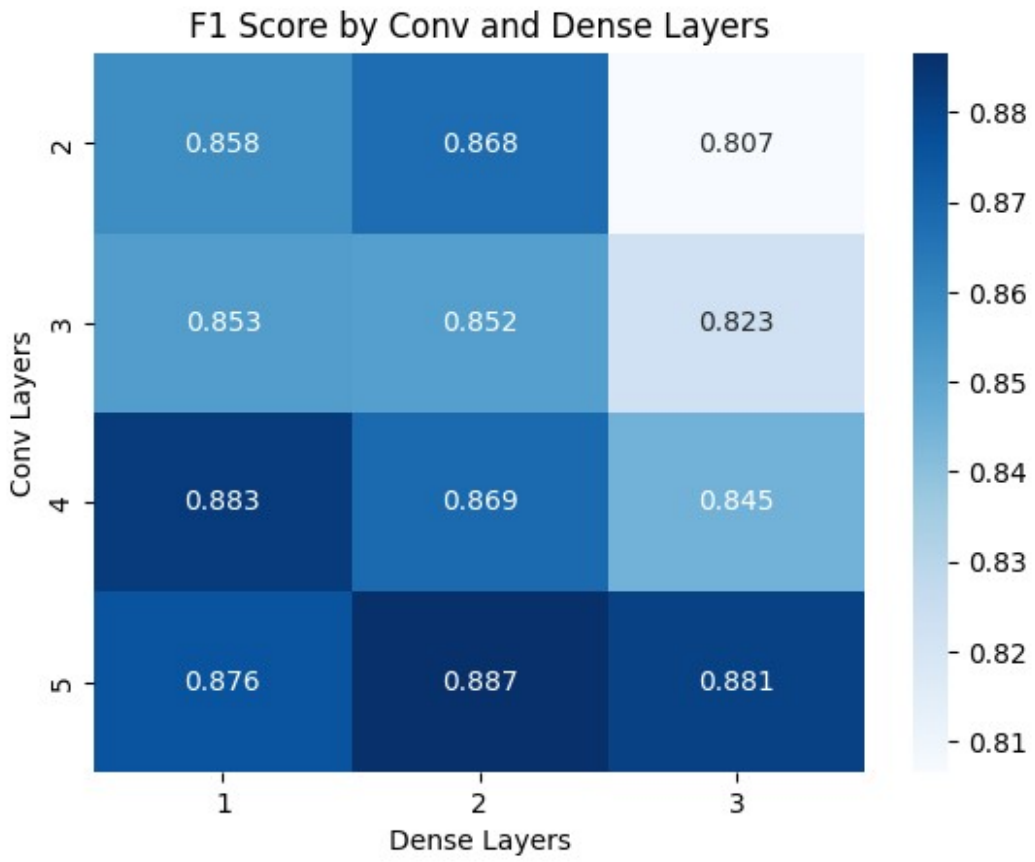
```
print("\n Average F1 Score by Batch Size:")
print(df_result.groupby("batch_size")
["f1_score"].mean().sort_values(ascending=False))

# Heatmap of F1 Score by conv_layers and dense_layers
pivot = df_result.pivot_table(values="f1_score",
index="num_conv_layers", columns="num_dense_layers")
print("\n Heatmap of F1 Score by Conv and Dense Layers:")
sns.heatmap(pivot, annot=True, cmap='Blues', fmt=".3f")
plt.title("F1 Score by Conv and Dense Layers")
plt.xlabel("Dense Layers")
plt.ylabel("Conv Layers")
plt.show()

# Boxplots for visual analysis
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
sns.boxplot(data=df_result, x="num_conv_layers", y="f1_score")
plt.title("F1 Score by Number of Conv Layers")

plt.subplot(1, 2, 2)
sns.boxplot(data=df_result, x="dropout_rate", y="f1_score")
plt.title("F1 Score by Dropout Rate")

plt.tight_layout()
plt.show()

# Total number of models
print(f"\nTotal number of models evaluated: {len(df_result)}")

# Best model config and its metrics
best_config = df_result.sort_values(by="f1_score",
ascending=False).iloc[0]
print("\n Best Configuration Overall:")
display(best_config.to_frame().T)
```

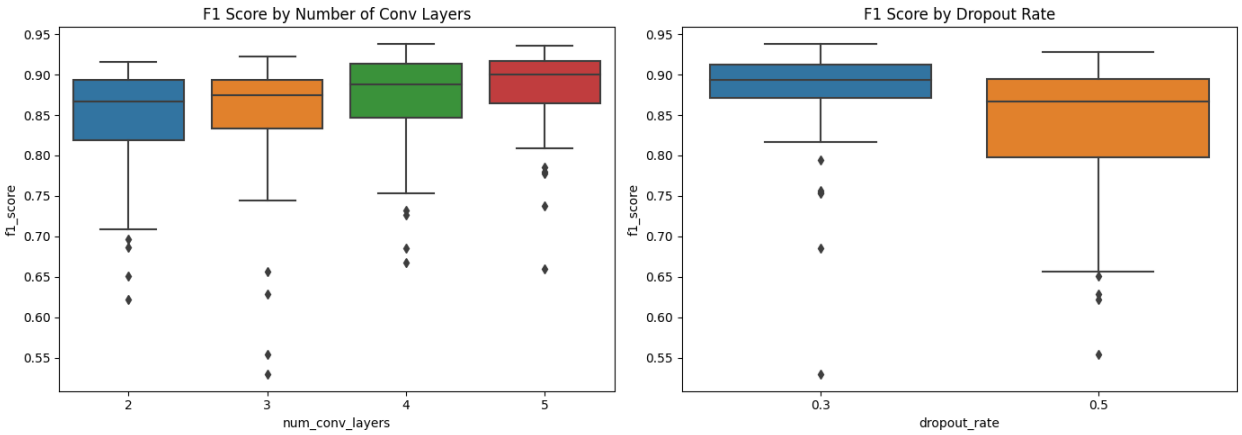
```
Top 10 Models by F1 Score:
```

	num_conv_layers	num_dense_layers	dropout_rate	batch_size	\
28	4	2	0.3	32	
172	4	2	0.3	32	
44	5	3	0.3	32	
93	5	3	0.3	16	
140	5	3	0.3	16	
37	5	1	0.3	32	
185	5	2	0.3	32	
170	4	1	0.5	32	
176	4	3	0.3	32	
120	4	1	0.3	16	

```
     conv_filters  dense_units  accuracy  precision    recall
f1_score  \
28              16          256  0.934169   0.938144  0.938144
0.938144
172             16          128  0.930251   0.913165  0.960236
0.936109
44              16          256  0.931818   0.935294  0.936672
0.935982
93              32          256  0.926332   0.937220  0.923417
0.930267
140             16          128  0.924765   0.923077  0.936672
0.929825
37              32          256  0.925549   0.935821  0.923417
0.929577
185             32          128  0.923981   0.934328  0.921944
0.928095
170             16          128  0.923981   0.936937  0.918999
0.927881
176             16          128  0.920846   0.909348  0.945508
0.927076
120             16          128  0.923981   0.951863  0.902798
0.926682

      roc_auc  training_time  \
28   0.983247          60.17
172  0.979416          60.40
44   0.981180          65.06
93   0.975518          87.96
140  0.977329          68.62
37   0.977677          79.71
185  0.975908          80.73
170  0.975839          58.72
176  0.976167          61.61
120  0.977689          62.19

                                                  history
28   {"accuracy": [0.7170588374137878, 0.8096078634...
172  {"accuracy": [0.70333331823349, 0.797450959682...
44   {"accuracy": [0.720588207244873, 0.82235294580...
93   {"accuracy": [0.7039215564727783, 0.8137254714...
140  {"accuracy": [0.6903921365737915, 0.7880392074...
37   {"accuracy": [0.7962744832038879, 0.8709803819...
185  {"accuracy": [0.7456862926483154, 0.8329411745...
170  {"accuracy": [0.7582352757453918, 0.8294117450...
176  {"accuracy": [0.6266666650772095, 0.7266666889...
120  {"accuracy": [0.773921549320221, 0.83666664361...


 Average F1 Score by Number of Convolutional Layers:
```

```
num_conv_layers
5    0.881203
4    0.865639
2    0.843935
3    0.842545
Name: f1_score, dtype: float64

 Average F1 Score by Number of Dense Layers:
num_dense_layers
2    0.868768
1    0.867166
3    0.839057
Name: f1_score, dtype: float64

 Average F1 Score by Dropout Rate:
dropout_rate
0.3    0.880460
0.5    0.836201
Name: f1_score, dtype: float64

 Average F1 Score by Convolutional Filters:
conv_filters
16    0.869521
32    0.847140
Name: f1_score, dtype: float64

 Average F1 Score by Batch Size:
batch_size
16    0.859768
32    0.856893
Name: f1_score, dtype: float64

 Heatmap of F1 Score by Conv and Dense Layers:
```






```
Total number of models evaluated: 192

 Best Configuration Overall:

   num_conv_layers num_dense_layers dropout_rate batch_size
conv_filters  \
28               4                 2          0.3         32
16
```

```
   dense_units  accuracy precision    recall  f1_score   roc_auc  \
28         256  0.934169  0.938144  0.938144  0.938144  0.983247

   training_time                                              history
28         60.17  {"accuracy": [0.7170588374137878, 0.8096078634...
```

Observations from the Results

- **Dropout rate**: A rate of **0.3** consistently outperformed 0.5.
  This lighter regularization allowed the network to retain more learned features while still mitigating overfitting.
- **Convolutional layers**: Architectures with **4 or 5 convolutional layers** performed best.
  Deeper networks extracted more hierarchical and abstract features, which improved classification performance.
- **Dense layers**: The best results were achieved with **2 dense layers**.
  A single dense layer may be too simplistic, while more than two could overfit or introduce unnecessary parameters.
- **Convolutional filters**: Using **16 base filters** (with progressive doubling) generally yielded slightly better results than 32.
  Starting with fewer filters reduced overfitting and computational cost. Since filters double in deeper layers, 16 was sufficient for learning without overwhelming the model or the GPU memory.
- **Dense units**: Configurations with **256 units** in the first dense layer (and halved thereafter) led to superior F1-scores compared to 128.
  More neurons gave the model better capacity to map complex, high-dimensional features extracted by the convolutional blocks.
- **Batch size**: A **batch size of 32** performed marginally better than 16.

3. Feature-Based Classification using a Pretrained CNN

In this section, we prepare a set of pretrained convolutional neural networks (CNNs) to be used as fixed feature extractors. Instead of training the CNNs from scratch, we leverage their pretrained weights (e.g., from ImageNet) to extract high-level image features.

The dictionary below maps:

- Model names → Keras applications (e.g., ResNet50, VGG16)
- Their associated preprocessing functions → required to format input data correctly for each model

These extracted features will later be used as input to traditional machine learning classifiers like Logistic Regression, K-Nearest Neighbors, SVM, Random Forest, Bagging and AdaBoost.

```
# Define a dictionary of available models and corresponding preprocess functions
cnn_models = {
    'ResNet50': (ResNet50, resnet_preprocess),
    'VGG16': (VGG16, vgg_preprocess),
    'EfficientNetB0': (EfficientNetB0, efficientnet_preprocess),
    'MobileNetV2': (MobileNetV2, mobilenet_preprocess),
    'InceptionV3': (InceptionV3, inception_preprocess)
}

cnn_names = list(cnn_models.keys())

def extract_features(model_name, input_images):
    print(f"\nExtracting features using {model_name}...")

    # Load the model and preprocessing function
    ModelClass, preprocess_func = cnn_models[model_name]

    # Build the model (without top classification layer)
    base_model = ModelClass(weights='imagenet', include_top=False, input_shape=(224, 224, 3), pooling='avg')
    feature_extractor = Model(inputs=base_model.input, outputs=base_model.output)

    # Preprocess images according to model
    images_preprocessed = preprocess_func(input_images.copy())

    # Extract features
    features = feature_extractor.predict(images_preprocessed, batch_size=32, verbose=1)

    print(f"{model_name} extracted feature shape: {features.shape}")
    return features
```

Logistic Regression on Extracted CNN Features

We train a **Logistic Regression classifier** on the feature vectors extracted from various pretrained CNN models.

What We Do:

- Define a range of regularization strengths via `C_values`
- For each CNN feature extractor:
  - Extract features from the dataset
  - Split features into training and test sets

  - Train logistic regression models across different `C` values
  - Evaluate using **F1-score**
- Save results for each CNN into a CSV

What We Visualize:

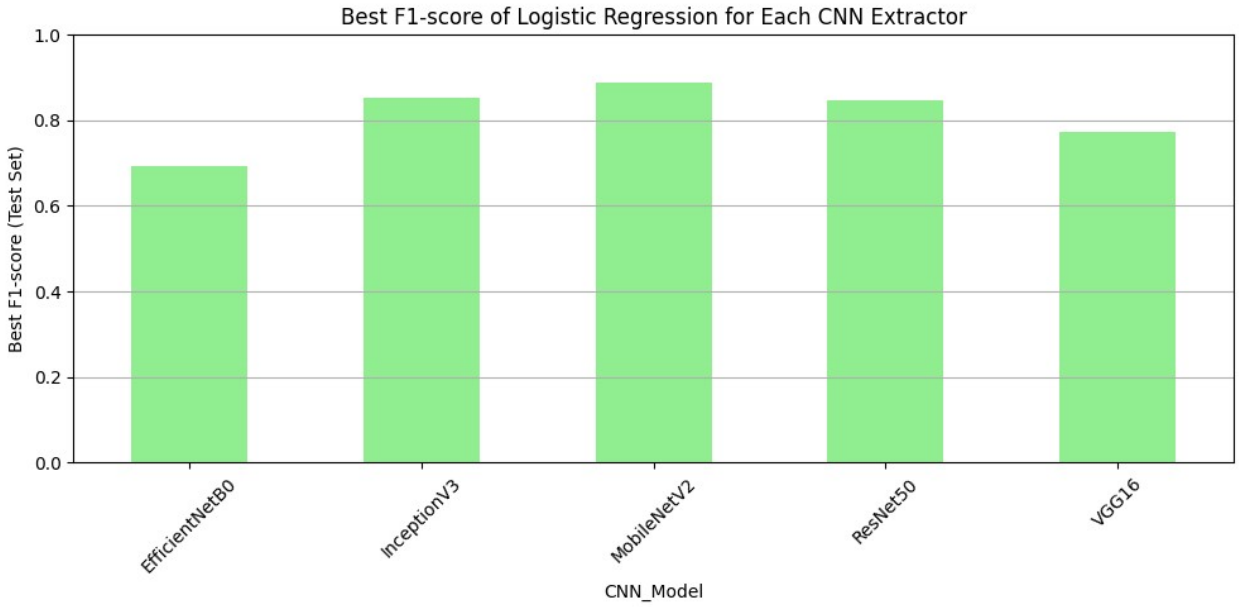
- **Best F1-score per CNN**: helps identify which feature extractor is most effective for this task
- **F1-score vs log10(C)**: reveals how sensitive each model is to regularization strength

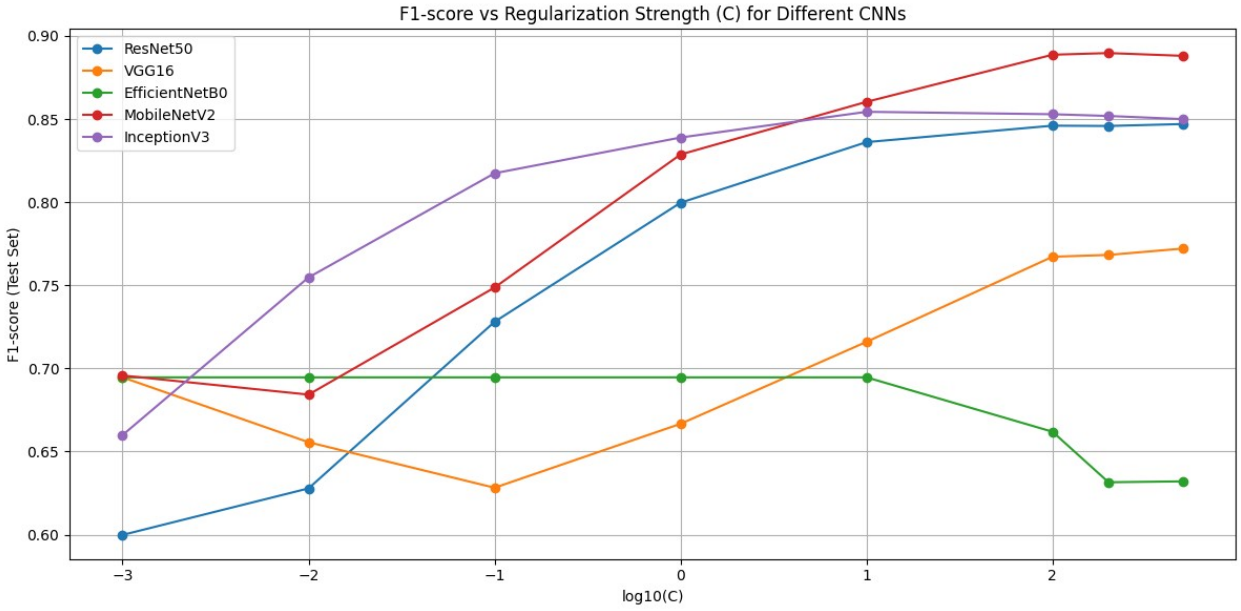
```
# Parameters
C_values = [0.001, 0.01, 0.1, 1, 10, 100, 200, 500]

# Logistic Regression Evaluation
results = []  # List to store the results before saving

for model_name in cnn_names:
    print(f"\n=== Logistic Regression on {model_name} ===")

    # Extract features
    features = extract_features(model_name, images)

    # Train/Test split
    X_train, X_test, y_train, y_test = train_test_split(
        features, labels, test_size=0.2, stratify=labels,
random_state=42
    )

    for C in C_values:
        model = LogisticRegression(C=C, max_iter=1000,
random_state=42)
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        f1 = f1_score(y_test, y_pred)
        results.append({
            'CNN_Model': model_name,
            'C_value': C,
            'F1_score': f1
        })
        del model

# Save results to CSV
results_df = pd.DataFrame(results)

# Save inside /kaggle/working
csv_save_path = '/kaggle/working/logreg_results.csv'
results_df.to_csv(csv_save_path, index=False)

print(f"\n Logistic Regression results saved successfully to
'{csv_save_path}'")
```

```
# Load results
logreg_results =
pd.read_csv('/kaggle/input/pretrain-results/results/results/logreg_res
ults.csv')

# Plot 1: Best F1-score per CNN
best_f1_per_cnn = logreg_results.groupby('CNN_Model')
['F1_score'].max()

# Find row with overall best
best_row = logreg_results.loc[logreg_results['F1_score'].idxmax()]

cnn = best_row['CNN_Model']
f1 = best_row['F1_score']

"""
# Append to CSV
pd.DataFrame([{
    'Method': 'Logistic Regression',
    'CNN_Model': cnn,
    'F1_score': f1
}]).to_csv(best_f1_csv_path, mode='a', header=False, index=False)
"""

print(f"[Logistic Regression] Best CNN: {best_row['CNN_Model']}, F1 =
{best_row['F1_score']:.4f}, C = {best_row['C_value']}")

plt.figure(figsize=(10, 5))
best_f1_per_cnn.plot(kind='bar', color='lightgreen')
plt.ylabel('Best F1-score (Test Set)')
plt.title('Best F1-score of Logistic Regression for Each CNN
Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Plot 2: F1-score vs log10(C) for each CNN
plt.figure(figsize=(12, 6))

for model_name in cnn_names:
    model_data = logreg_results[logreg_results['CNN_Model'] ==
model_name]
    plt.plot(np.log10(model_data['C_value']), model_data['F1_score'],
marker='o', label=model_name)

plt.xlabel('log10(C)')
plt.ylabel('F1-score (Test Set)')
plt.title('F1-score vs Regularization Strength (C) for Different
CNNs')
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
[Logistic Regression] Best CNN: MobileNetV2, F1 = 0.8897, C = 200.0
```





Observations from Logistic Regression Results

- **Best overall CNN extractor**: **MobileNetV2** achieved the highest F1-score (~0.89) with `C = 200`.

- **InceptionV3** also performed competitively, especially at moderate regularization strengths, showing its ability to generalize across varying C values.
- **EfficientNetB0** and **VGG16** exhibited lower F1-scores overall, suggesting that their extracted features may not be as linearly separable in this task context.
- **ResNet50** performed moderately well, but did not reach the peak F1-scores achieved by MobileNetV2 or InceptionV3.

- In the second plot (F1-score vs log10(C)):
  - Most models **improved** as regularization strength **decreased** (i.e., larger C), showing that allowing more model flexibility helped.
  - MobileNetV2 benefited the most from higher C values, indicating it has more complex and informative features that require weaker regularization.
  - EfficientNetB0 and VGG16 showed **less sensitivity** to C values, remaining relatively flat or unstable across the range.

These results suggest that **MobileNetV2 is a robust and efficient feature extractor** when combined with simple linear classifiers like logistic regression.

K-Nearest Neighbors on Extracted CNN Features

We apply the K-Nearest Neighbors (KNN) algorithm on the feature vectors extracted from various pretrained CNN models.

What We Do:

- Define a range of neighbor values via `K_values`
- Use two distance metrics: `'euclidean'` and `'cosine'`
- For each CNN feature extractor:
  - Extract features from the dataset
  - Split features into training and test sets
  - Normalize features if using cosine distance
  - Train KNN models for each `(K, metric)` pair
  - Evaluate using F1-score
- Save results for each configuration into a CSV

What We Visualize:

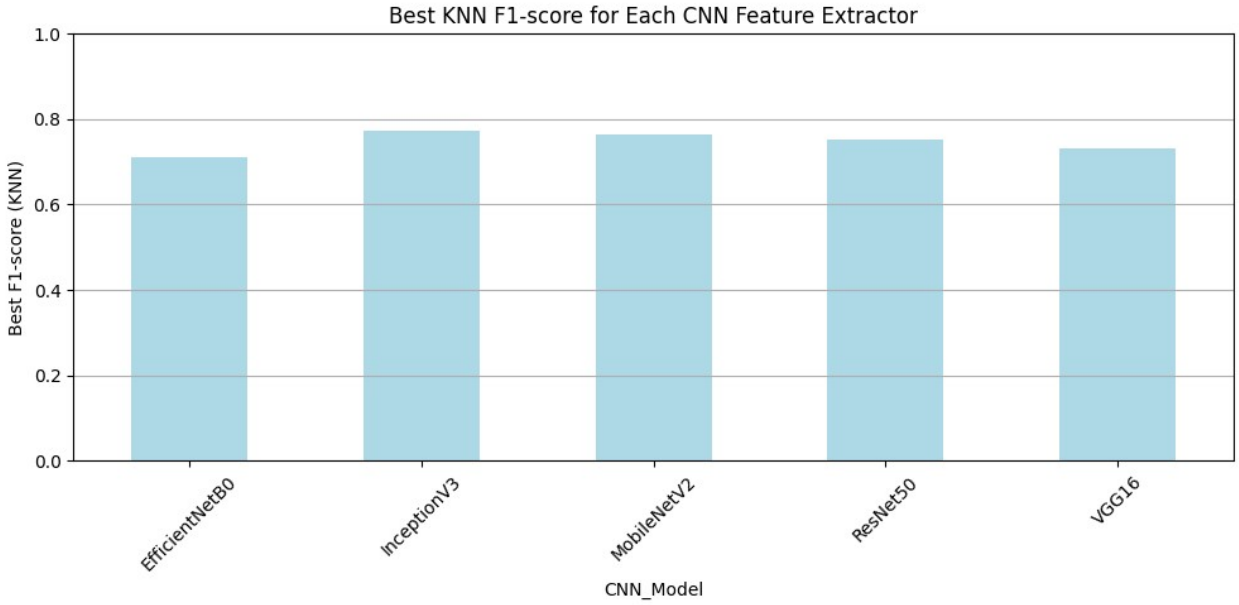
- **Best F1-score per CNN**: shows which CNN feature extractor enables the most effective KNN classification
- **F1-score vs K (for each metric and CNN)**: reveals how performance varies with neighborhood size and distance type

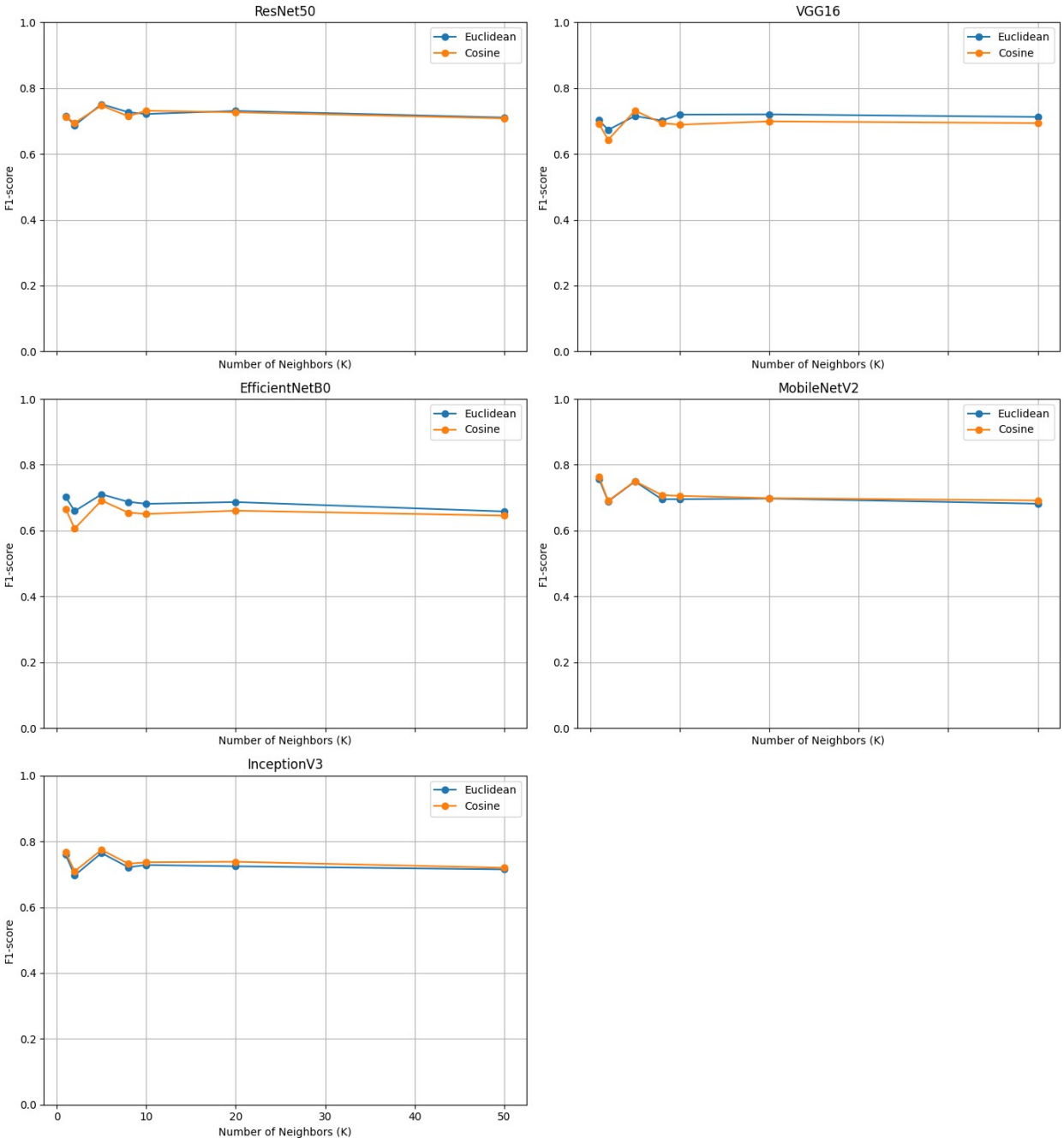
```
# Parameters
K_values = [1, 2, 5, 8, 10, 20, 50]
distance_metrics = ['euclidean', 'cosine']

# K-Nearest Neighbors Evaluation
knn_results = []

for model_name in cnn_names:
    print(f"\n=== KNN on {model_name} ===")
    features = extract_features(model_name, images)
    X_train, X_test, y_train, y_test = train_test_split(
        features, labels, test_size=0.2, stratify=labels,
random_state=42
    )

    for metric in distance_metrics:
        print(f"-- Metric: {metric}")

        if metric == 'cosine':
            X_train_used = normalize(X_train, norm='l2')
            X_test_used = normalize(X_test, norm='l2')
        else:
            X_train_used = X_train
            X_test_used = X_test

        for k in K_values:
            knn = KNeighborsClassifier(n_neighbors=k, metric=metric)
            knn.fit(X_train_used, y_train)
            y_pred = knn.predict(X_test_used)
            f1 = f1_score(y_test, y_pred)
            knn_results.append({
                'CNN_Model': model_name,
                'Distance_Metric': metric,
                'K_value': k,
                'F1_score': f1
            })
            del knn

# Save results to CSV
knn_df = pd.DataFrame(knn_results)
csv_path = '/kaggle/working/knn_results.csv'
knn_df.to_csv(csv_path, index=False)

print(f"\n KNN results saved to '{csv_path}'")
```

```
# Load KNN results
#knn_df = pd.read_csv('/kaggle/working/knn_results.csv')
knn_df =
pd.read_csv('/kaggle/input/pretrain-results/results/results/knn_result
s.csv')

# Define distance metrics
distance_metrics = ['euclidean', 'cosine']

# Plot 1: Best F1-score per CNN
best_knn_scores = knn_df.groupby('CNN_Model')['F1_score'].max()

# Print the overall best configuration
best_row = knn_df.loc[knn_df['F1_score'].idxmax()]

cnn = best_row['CNN_Model']
f1 = best_row['F1_score']

"""
pd.DataFrame([{
    'Method': 'KNN',
    'CNN_Model': cnn,
    'F1_score': f1
}]).to_csv(best_f1_csv_path, mode='a', header=False, index=False)
"""

print(f"[KNN] Best CNN: {best_row['CNN_Model']}, F1 =
{best_row['F1_score']:.4f}, K = {best_row['K_value']}, Metric =
{best_row['Distance_Metric']}")

# Bar plot showing best F1 per CNN
plt.figure(figsize=(10, 5))
best_knn_scores.plot(kind='bar', color='lightblue')
plt.ylabel('Best F1-score (KNN)')
plt.title('Best KNN F1-score for Each CNN Feature Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Grid size: 2 columns, enough rows to cover all models
n_models = len(cnn_names)
n_cols = 2
n_rows = (n_models + 1) // 2  # Round up

# Create subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 5 * n_rows),
sharex=True)

# Flatten axes array for easy indexing
```

```
axes = axes.flatten()

# Plot each CNN's curves
for i, model_name in enumerate(cnn_names):
    ax = axes[i]
    for metric in distance_metrics:
        subset = knn_df[(knn_df['CNN_Model'] == model_name) &
(knn_df['Distance_Metric'] == metric)]
        ax.plot(subset['K_value'], subset['F1_score'], marker='o',
label=metric.capitalize())

    ax.set_title(f"{model_name}")
    ax.set_xlabel("Number of Neighbors (K)")
    ax.set_ylabel("F1-score")
    ax.set_ylim(0, 1)
    ax.grid(True)
    ax.legend()

# Turn off any unused subplot (e.g. the 6th one if only 5 models)
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```

```
[KNN] Best CNN: InceptionV3, F1 = 0.7744, K = 5, Metric = cosine
```





Observations from K-Nearest Neighbors (KNN) Results

- **Best overall CNN extractor**: **InceptionV3** achieved the highest F1-score (~0.7744) using **K = 5** and the **cosine** distance metric.
- **MobileNetV2** followed closely, showing stable performance across different K values and both distance metrics.
- **EfficientNetB0** and **VGG16** exhibited slightly lower F1-scores, especially under higher K values, suggesting their features may be less optimal for non-parametric classification in this context.

- **ResNet50** showed decent but not leading performance, with noticeable drops for large K.

In the subplots (F1-score vs Number of Neighbors):

- Most models peaked in performance for small K values (**K = 5 or 10**) and showed a **decline** as K increased, indicating that over-smoothing (averaging over too many neighbors) hurts performance.
- **Cosine distance** often matched or slightly outperformed **Euclidean distance**, especially for deeper models like InceptionV3 and MobileNetV2. This suggests that angular relationships between features are informative.
- **InceptionV3** displayed consistent performance across distance types and had the **smoothest curve**, reflecting robust and well-clustered feature representations.
- **EfficientNetB0** and **VGG16** showed greater sensitivity to K, indicating less stable feature embedding quality.

These results indicate that **InceptionV3** is a particularly effective feature extractor for KNN-based classification, especially when combined with cosine similarity and smaller neighbor counts.

SVM on Extracted CNN Features

We train a **Support Vector Machine (SVM)** classifier on the feature vectors extracted from various pretrained CNN models.

What We Do:

- Define a range of regularization strengths via `C_values`
- Explore multiple kernel types: `'linear'`, `'rbf'`, and `'cosine'`
- For each CNN feature extractor:
  - Extract features from the dataset
  - Split features into training and test sets
  - Train SVM classifiers using different kernels and `C` values
  - Evaluate using F1-score
- Save results for each configuration into a CSV

What We Visualize:

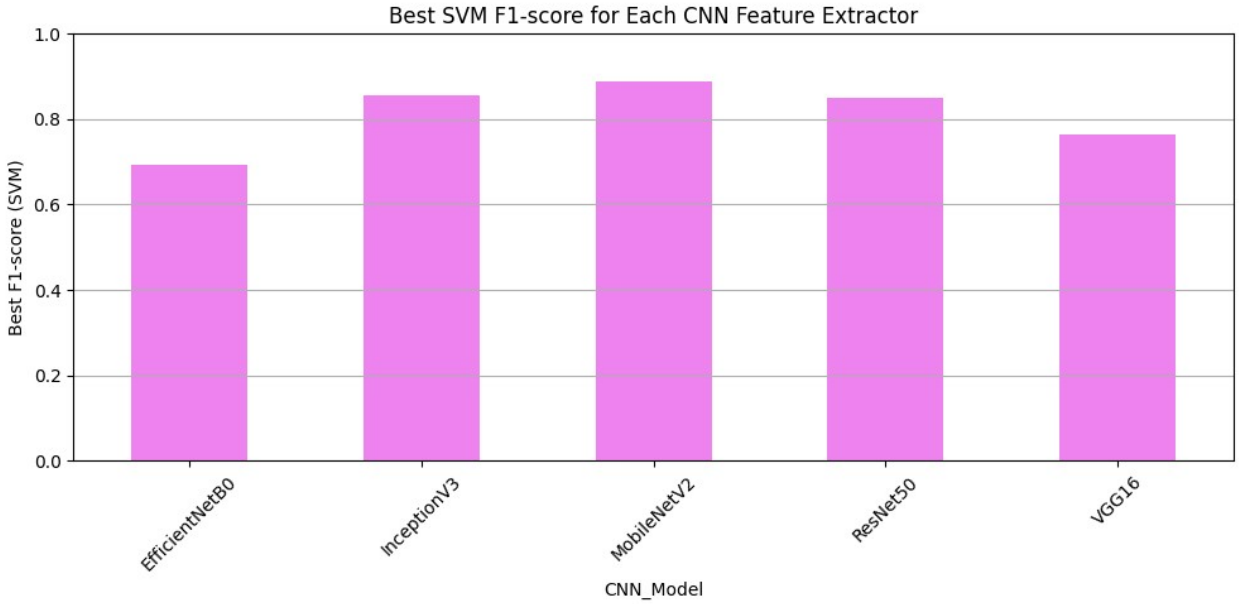
- **Best F1-score per CNN**: helps identify the most effective feature extractor when combined with SVM
- **F1-score vs log10(C)** for each kernel: reveals how sensitive each model is to regularization strength across different kernels

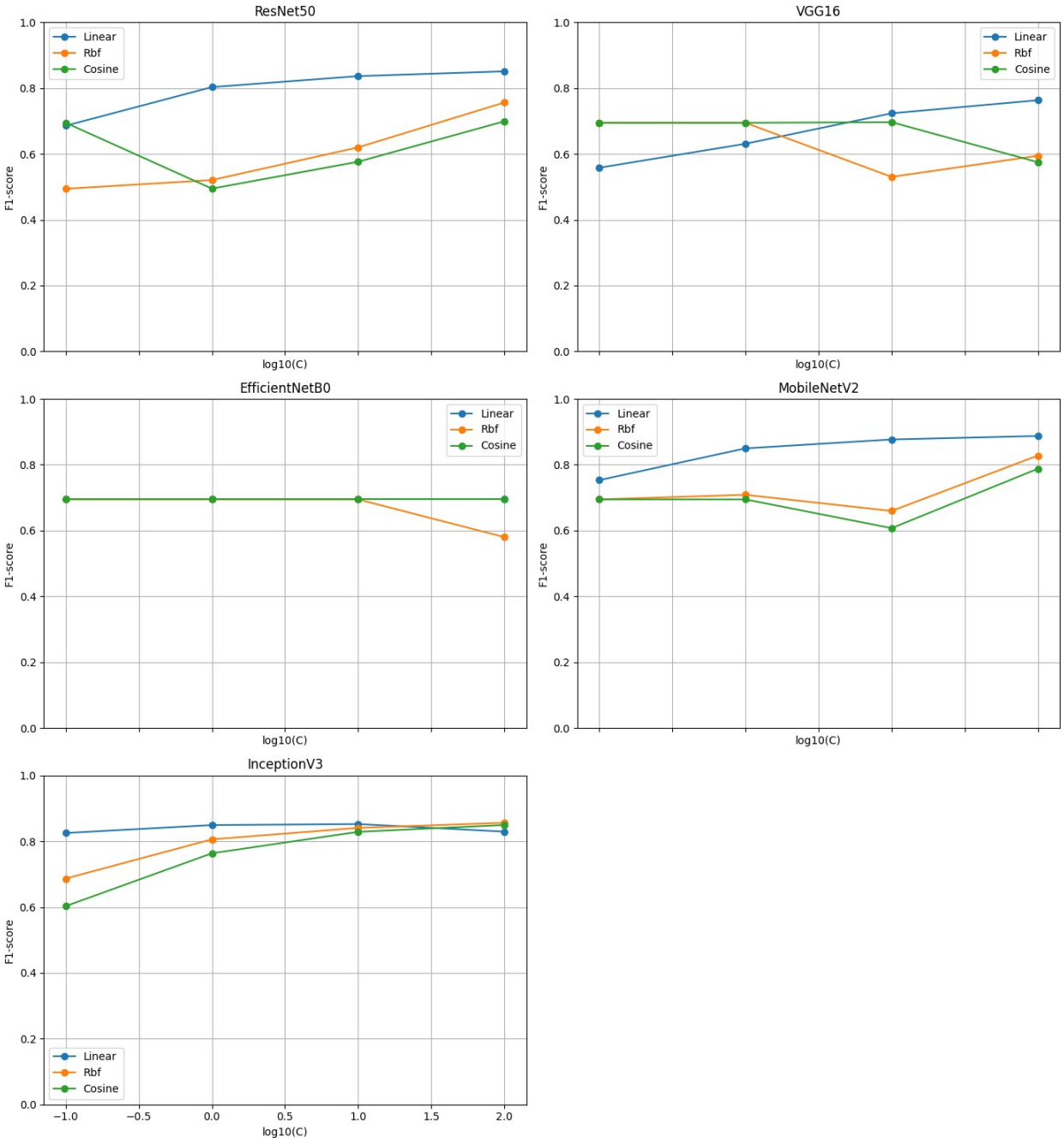
```
# Parameters
C_values = [0.1, 1, 10, 100]
kernels = ['linear', 'rbf', 'cosine']
svm_results = []

for model_name in cnn_names:
    print(f"\n=== SVM on {model_name} ===")
    features = extract_features(model_name, images)
    X_train, X_test, y_train, y_test = train_test_split(
        features, labels, test_size=0.2, stratify=labels,
random_state=42
    )

    for kernel in kernels:
        print(f"-- Kernel: {kernel}")
        for C in C_values:
            if kernel == 'cosine':
                X_train_norm = normalize(X_train, norm='l2')
                X_test_norm = normalize(X_test, norm='l2')
                K_train = cosine_similarity(X_train_norm)
                K_test = cosine_similarity(X_test_norm, X_train_norm)
                model = SVC(C=C, kernel='precomputed')
                model.fit(K_train, y_train)
                y_pred = model.predict(K_test)
            else:
                model = SVC(C=C, kernel=kernel)
                model.fit(X_train, y_train)
                y_pred = model.predict(X_test)

            f1 = f1_score(y_test, y_pred)
            svm_results.append({
                'CNN_Model': model_name,
                'Kernel': kernel,
                'C_value': C,
                'F1_score': f1
            })

            del model
            gc.collect()

# Save results to CSV
svm_df = pd.DataFrame(svm_results)
svm_df.to_csv('/kaggle/working/svm_results.csv', index=False)
print("\n SVM results saved to /kaggle/working/svm_results.csv")

# --- Load SVM results ---
#svm_df = pd.read_csv('/kaggle/working/svm_results.csv')
svm_df =
pd.read_csv('/kaggle/input/pretrain-results/results/results/svm_result
s.csv')
```

```
# --- Plot 1: Best F1-score per CNN ---
best_svm_scores = svm_df.groupby('CNN_Model')['F1_score'].max()

# Find best row
best_row = svm_df.loc[svm_df['F1_score'].idxmax()]

cnn = best_row['CNN_Model']
f1 = best_row['F1_score']

"""
pd.DataFrame([{
    'Method': 'SVM',
    'CNN_Model': cnn,
    'F1_score': f1
}]).to_csv(best_f1_csv_path, mode='a', header=False, index=False)
"""

print(f"[SVM] Best CNN: {best_row['CNN_Model']}, F1 =
{best_row['F1_score']:.4f}, Kernel = {best_row['Kernel']}, C =
{best_row['C_value']}")

plt.figure(figsize=(10, 5))
best_svm_scores.plot(kind='bar', color='violet')
plt.ylabel('Best F1-score (SVM)')
plt.title('Best SVM F1-score for Each CNN Feature Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

kernels = ['linear', 'rbf', 'cosine']

# Grid: 2 columns, calculate rows
n_models = len(cnn_names)
n_cols = 2
n_rows = (n_models + 1) // 2

# Create subplots
fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 5 * n_rows),
sharex=True)
axes = axes.flatten()

# Plot for each CNN model
for i, model_name in enumerate(cnn_names):
    ax = axes[i]
    for kernel in kernels:
        subset = svm_df[(svm_df['CNN_Model'] == model_name) &
(svm_df['Kernel'] == kernel)]
```

```
        ax.plot(np.log10(subset['C_value']), subset['F1_score'],
marker='o', label=kernel.capitalize())

    ax.set_title(f"{model_name}")
    ax.set_xlabel("log10(C)")
    ax.set_ylabel("F1-score")
    ax.set_ylim(0, 1)
    ax.grid(True)
    ax.legend()

# Turn off unused subplot if models < grid size
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```

```
[SVM] Best CNN: MobileNetV2, F1 = 0.8874, Kernel = linear, C = 100.0
```





Observations from SVM Results

Best overall CNN extractor: MobileNetV2 achieved the highest F1-score (~0.887) with a **linear kernel** and **C = 100**.

InceptionV3 also performed strongly, especially with higher C values across all kernels, indicating robustness and effective representation learning.

EfficientNetB0 consistently produced lower F1-scores, particularly with RBF, suggesting it may be less suited for linear separation in this task.

ResNet50 benefited most from the linear kernel, with F1 improving steadily with larger C values.

VGG16 showed moderate performance, with the linear kernel outperforming others, and a relatively stable score across different C values.

From the kernel comparison plots (F1-score vs log10(C)):

- **Linear kernels** generally outperformed RBF and Cosine across most CNNs, showing that the extracted features are often linearly separable.
- **Cosine kernel** remained flat in performance for most models (e.g., EfficientNetB0, VGG16), indicating limited sensitivity to C values.
- **InceptionV3 and MobileNetV2** exhibited increasing performance with larger C, especially with linear and RBF kernels, emphasizing their adaptability under less regularization.
- **EfficientNetB0 with RBF** showed a drop in performance at higher C, indicating potential overfitting or poor compatibility with nonlinear boundaries.

These results support that **MobileNetV2** and **InceptionV3** are effective feature extractors for SVM classification, particularly when paired with linear kernels and appropriately tuned regularization.

Random Forest on Extracted CNN Features

We train a **Random Forest classifier** on the feature vectors extracted from pretrained CNN models.

What We Do:

- Define grid search over:
  - Number of trees (`n_estimators`): 50, 100, 200
  - Maximum tree depth (`max_depth`): 10, 20, ∞ (None)
- For a selected CNN feature extractor:
  - Extract feature vectors from the dataset
  - Split data into training and test sets
  - Train Random Forest classifiers for all (n_estimators, max_depth) combinations
  - Evaluate each configuration using **F1-score**
- Save the results into a CSV file for further analysis

What We Visualize:

- **F1-score vs Number of Trees**: for each `max_depth`, showing how increasing model complexity impacts performance
- **Heatmap of F1-score** across trees and depth: helps highlight the optimal configuration for this CNN feature extractor

---

Note: The Random Forest training was executed separately for each CNN (i.e., `cnn_names[0]` to `cnn_names[4]`), because running all CNNs in a single loop caused GPU memory exhaustion, leading to session crashes.

```
model_name = cnn_names[0]
print(f"\n=== Random Forest on CNN: {model_name} ===")

randomforest_results = []
rf_n_estimators = [50, 100, 200]
rf_max_depths = [None, 10, 20]

features = extract_features(model_name, images)
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.2, stratify=labels, random_state=42
)

for n in rf_n_estimators:
    for depth in rf_max_depths:
        print(f"[Random Forest] CNN={model_name} | n_estimators={n} |
max_depth={depth}")
        model = RandomForestClassifier(n_estimators=n,
max_depth=depth, random_state=42)
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        f1 = f1_score(y_test, y_pred)
        randomforest_results.append({
            'CNN_Model': model_name,
            'Method': 'Random Forest',
            'Param_1': f'n_estimators={n}',
            'Param_2': f'max_depth={depth}',
            'F1_score': f1
        })
        del model
        gc.collect()

df = pd.DataFrame(randomforest_results)
df.to_csv('/kaggle/working/randomforest_results.csv', mode='a',
header=not os.path.exists('/kaggle/working/randomforest_results.csv'),
index=False)
```

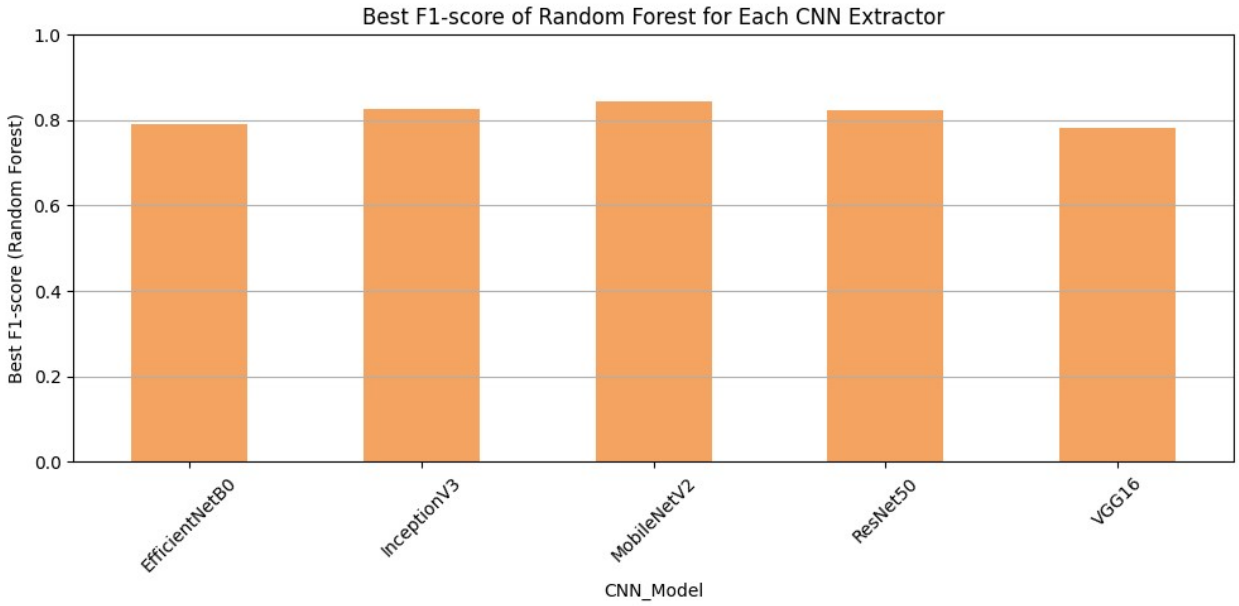
```
=== Random Forest on CNN: ResNet50 ===
```

```
 Extracting features using ResNet50...
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/resnet/
resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_784/685517338.py in <cell line: 0>()
      6 rf_max_depths = [None, 10, 20]
      7
----> 8 features = extract_features(model_name, images)
      9 X_train, X_test, y_train, y_test = train_test_split(
     10     features, labels, test_size=0.2, stratify=labels,
random_state=42

/tmp/ipykernel_784/3073812736.py in extract_features(model_name,
input_images)
     16
     17     # Build the model (without top classification layer)
---> 18     base_model = ModelClass(weights='imagenet',
include_top=False, input_shape=(224, 224, 3), pooling='avg')
     19     feature_extractor = Model(inputs=base_model.input,
outputs=base_model.output)
     20

/usr/local/lib/python3.11/dist-packages/keras/src/applications/resnet.
py in ResNet50(include_top, weights, input_tensor, input_shape,
pooling, classes, classifier_activation, name)
    407         return stack_residual_blocks_v1(x, 512, 3,
name="conv5")
    408
--> 409     return ResNet(
    410         stack_fn,
    411         preact=False,

/usr/local/lib/python3.11/dist-packages/keras/src/applications/resnet.
py in ResNet(stack_fn, preact, use_bias, include_top, weights,
input_tensor, input_shape, pooling, classes, classifier_activation,
name, weights_name)
    210             file_hash=file_hash,
    211         )
--> 212         model.load_weights(weights_path)
    213     elif weights is not None:
    214         model.load_weights(weights)

/usr/local/lib/python3.11/dist-packages/keras/src/utils/traceback_util
```

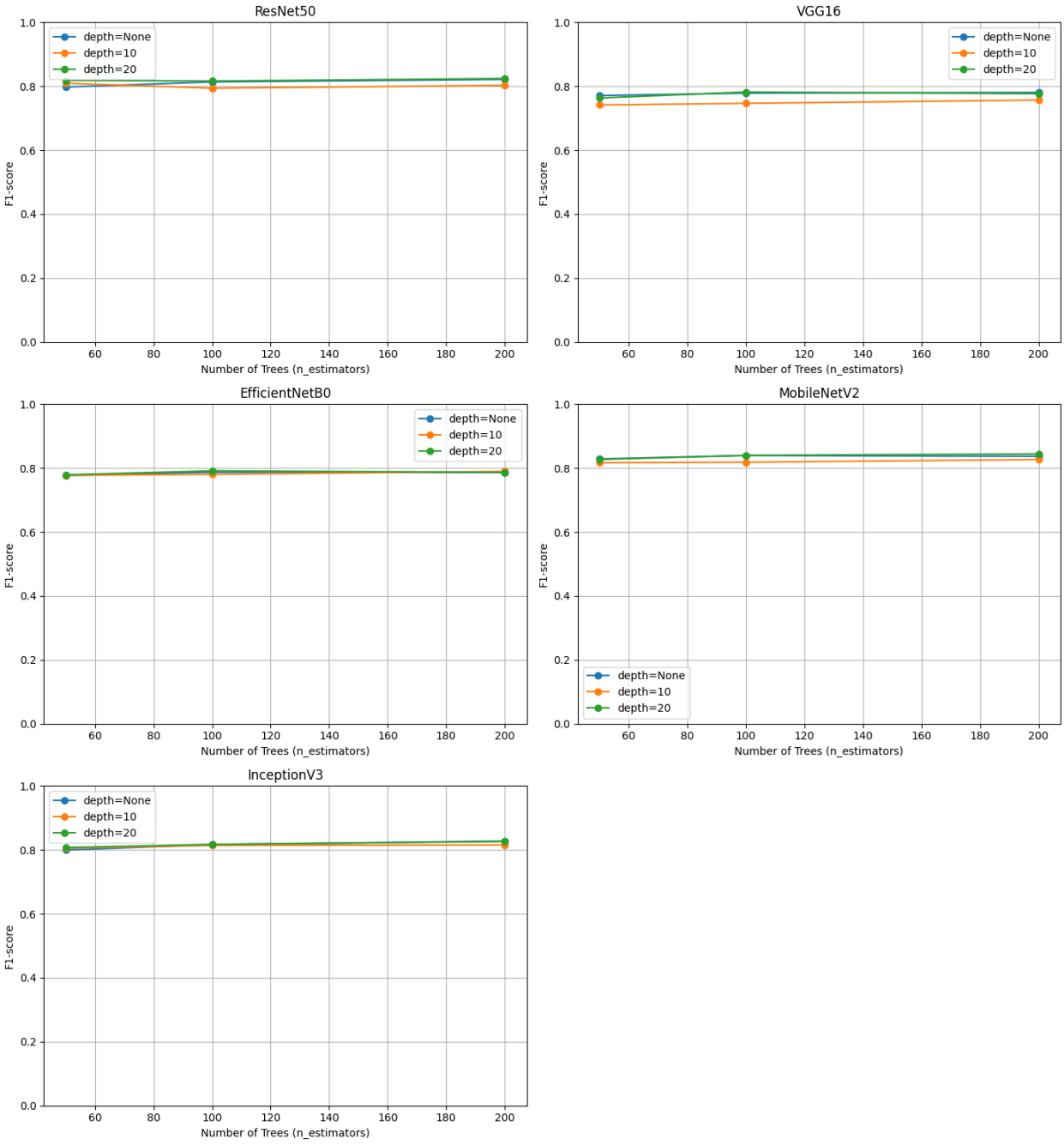
```
s.py in error_handler(*args, **kwargs)
    115         filtered_tb = None
    116         try:
--> 117             return fn(*args, **kwargs)
    118         except Exception as e:
    119             filtered_tb =
_process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/keras/src/models/model.py in
load_weights(self, filepath, skip_mismatch, **kwargs)
    351                     the shape of the weights.
    352         """
--> 353         saving_api.load_weights(
    354             self, filepath, skip_mismatch=skip_mismatch,
**kwargs
    355         )

/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_api.py
in load_weights(model, filepath, skip_mismatch, **kwargs)
    269                 )
    270             else:
--> 271
legacy_h5_format.load_weights_from_hdf5_group(f, model)
    272     else:
    273         raise ValueError(

/usr/local/lib/python3.11/dist-packages/keras/src/legacy/saving/legacy
_h5_format.py in load_weights_from_hdf5_group(f, model)
    374             )
    375         for ref_v, val in zip(symbolic_weights,
weight_values):
--> 376             ref_v.assign(val)
    377
    378     if "top_level_model_weights" in f:

/usr/local/lib/python3.11/dist-packages/keras/src/backend/common/varia
bles.py in assign(self, value)
    221         return self._maybe_autocast(self._value)
    222
--> 223     def assign(self, value):
    224         value = self._convert_to_tensor(value,
dtype=self.dtype)
    225         if not shape_equal(value.shape, self.shape):

KeyboardInterrupt:

# File paths
rf_path = '/kaggle/input/pretrain-results/results-ensembled/results-
ensembled/randomforest_results.csv'
best_f1_csv_path = '/kaggle/working/best_f1_summary.csv'
```

```
# Load data
rf_df = pd.read_csv(rf_path)

# Step 1: Best F1-score per CNN
best_rf_scores = rf_df.groupby('CNN_Model')['F1_score'].max()

# Step 2: Bar Plot (best per CNN)
plt.figure(figsize=(10, 5))
best_rf_scores.plot(kind='bar', color='sandybrown')
plt.ylabel('Best F1-score (Random Forest)')
plt.title('Best F1-score of Random Forest for Each CNN Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Step 3: Best overall configuration
best_rf_row = rf_df.loc[rf_df['F1_score'].idxmax()]
print(f"[Random Forest] Best CNN: {best_rf_row['CNN_Model']}, F1 =
{best_rf_row['F1_score']:.4f}, "
      f"Params: {best_rf_row['Param_1']}, {best_rf_row['Param_2']}")

"""
# Step 4: Append to summary CSV
pd.DataFrame([{
    'Method': 'Random Forest',
    'CNN_Model': best_rf_row['CNN_Model'],
    'F1_score': best_rf_row['F1_score']
}]).to_csv(best_f1_csv_path, mode='a', header=False, index=False)
"""

# Unique CNNs and max_depth values
cnn_names = rf_df['CNN_Model'].unique()
depth_values = rf_df['Param_2'].unique()

# Grid setup: 2 columns, enough rows
n_models = len(cnn_names)
n_cols = 2
n_rows = (n_models + 1) // 2

# Create subplot grid
fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 5 * n_rows),
sharex=False)
axes = axes.flatten()

# Loop through CNNs
for i, model_name in enumerate(cnn_names):
    ax = axes[i]
```

```
    for depth in depth_values:
        subset = rf_df[(rf_df['CNN_Model'] == model_name) &
(rf_df['Param_2'] == depth)]
        if subset.empty:
            continue
        # Extract integer n_estimators from strings like
'n_estimators=100'
        n_vals = subset['Param_1'].str.extract(r'n_estimators=(\
d+)').astype(int)[0]
        ax.plot(n_vals, subset['F1_score'], marker='o',
label=f"depth={depth.split('=')[1]}")

    ax.set_title(f"{model_name}")
    ax.set_xlabel("Number of Trees (n_estimators)")
    ax.set_ylabel("F1-score")
    ax.set_ylim(0, 1)
    ax.grid(True)
    ax.legend()

# Hide any unused subplot
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```




```
[Random Forest] Best CNN: MobileNetV2, F1 = 0.8443, Params:
n_estimators=200, max_depth=20
```



Observations from Random Forest Results

Best overall CNN extractor: MobileNetV2 achieved the highest F1-score (~0.8443) with **200 trees** and **maximum depth of 20**.

- InceptionV3 also performed competitively, producing stable and strong results across all tested configurations.
- ResNet50 and EfficientNetB0 delivered moderate performance, with F1-scores slightly lower than MobileNetV2 and InceptionV3.

- VGG16 consistently trailed in performance, suggesting its extracted features may be less discriminative for this task.

From the parameter comparison plots (F1-score vs n_estimators and max_depth):

- **More trees helped performance**: Increasing `n_estimators` generally resulted in improved or stable F1-scores across all models, with **200 trees** consistently yielding the best results.
- **Deeper trees led to higher F1**: A `max_depth` of **20** showed marginal improvements over depth=10 and no depth limit (None), likely due to better feature partitioning.
- **Shallow trees underperform**: Models with `max_depth=10` often produced slightly lower F1-scores, especially for more complex feature spaces.
- **Model stability**: InceptionV3 and MobileNetV2 showed stable performance across different tree depths and counts, highlighting their robust feature representations.

These findings suggest that **MobileNetV2** and **InceptionV3** are the most suitable CNN feature extractors when using Random Forests, particularly when paired with **deeper trees** and **a higher number of estimators**.

Bagging on Extracted CNN Features

We train a **Bagging ensemble model** using a **Linear Support Vector Machine (SVM)** as the base estimator on features extracted from various pretrained CNN models.

What We Do:

- Define a grid search over:
  - Number of estimators (**n_estimators**): 10, 15, 20
- For a selected CNN feature extractor:
  - Extract feature vectors from the dataset
  - Split data into training and test sets
  - Train a Bagging classifier with a Linear SVM base for each `n_estimators` configuration
  - Evaluate each configuration using **F1-score**
- Save the results into a CSV file for downstream analysis

What We Visualize:

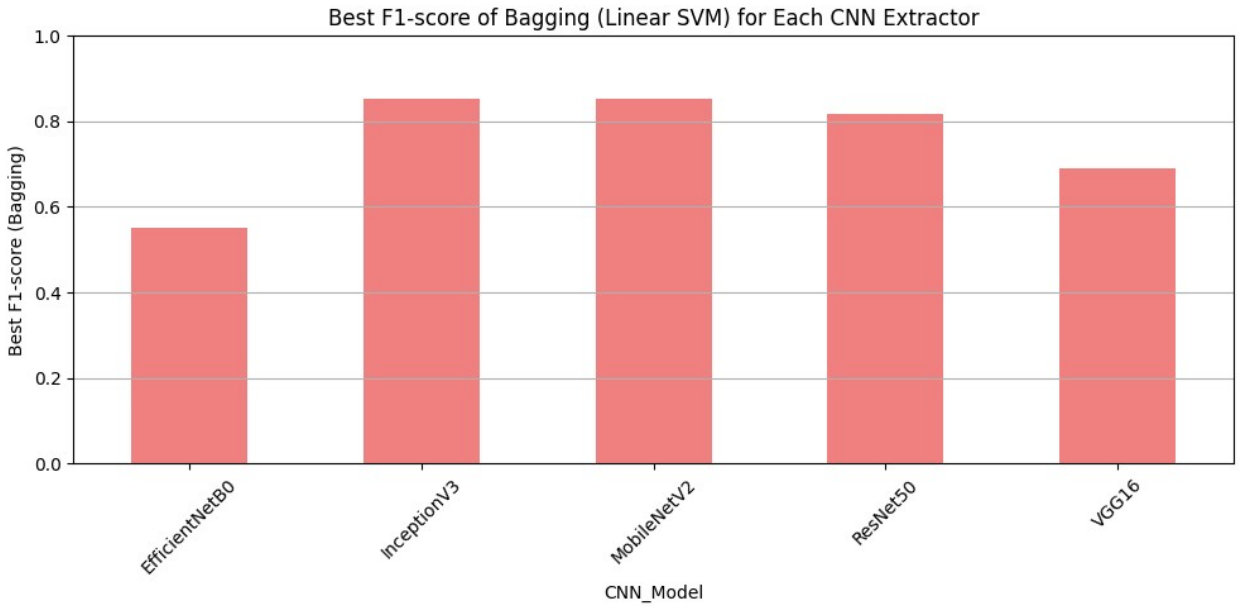
- **Best F1-score per CNN**: allows us to compare which CNN backbone performs best when paired with a bagging ensemble
- **F1-score vs Number of Estimators**: for each CNN, showing how ensemble size affects performance

---

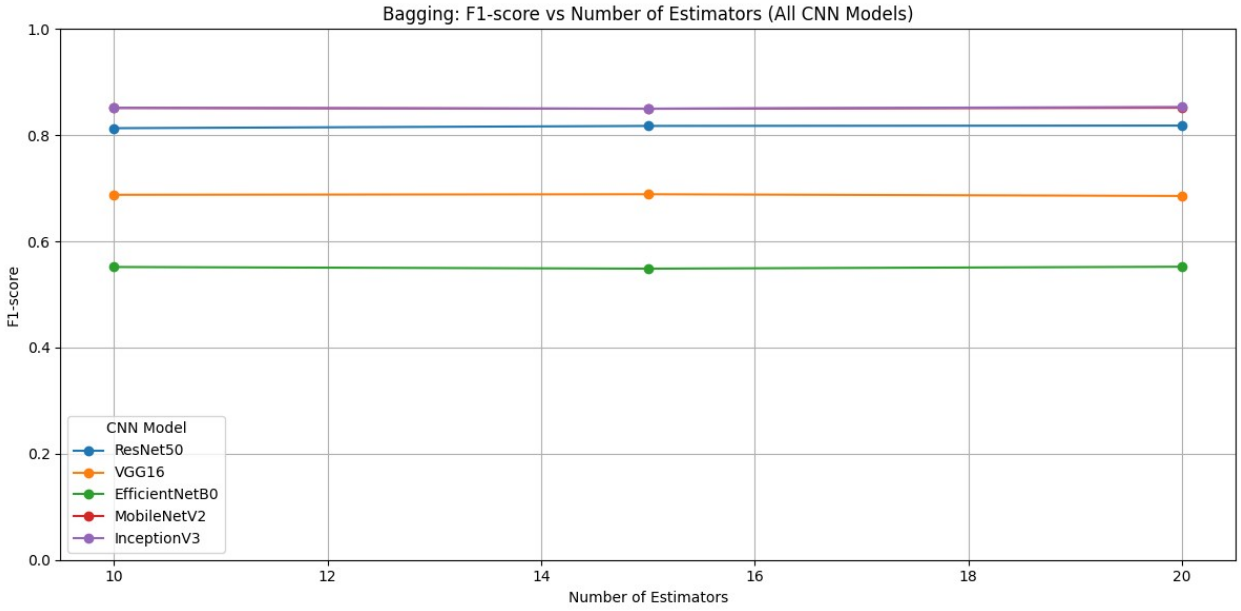
Note: Bagging training was executed separately for each CNN (i.e., `cnn_names[0]` to `cnn_names[4]`) because running all configurations in a single session exhausted GPU memory, leading to session crashes. This approach ensured training stability and consistent results across all configurations.

```
bagging_n_estimators = [10, 15, 20]

model_name = cnn_names[0]
print(f"\n=== Bagging (SVM) on CNN: {model_name} ===")
bagging_results = []

features = extract_features(model_name, images)
X_train, X_test, y_train, y_test = train_test_split(features, labels,
test_size=0.2, stratify=labels, random_state=42)

for n in bagging_n_estimators:
    print(f"[Bagging] CNN={model_name} | n_estimators={n} | base=SVM")
    base_svm = SVC(kernel='linear', probability=True, random_state=42)
    model = BaggingClassifier(base_estimator=base_svm, n_estimators=n,
random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred)
    bagging_results.append({
        'CNN_Model': model_name,
        'Method': 'Bagging (Linear SVM)',
        'Param_1': f'n_estimators={n}',
        'Param_2': 'base=SVM',
        'F1_score': f1
    })
    del model
    gc.collect()

df = pd.DataFrame(bagging_results)
df.to_csv('/kaggle/working/bagging_results.csv', mode='a', header=not
os.path.exists('/kaggle/working/bagging_results.csv'), index=False)

# Paths
bag_path ='/kaggle/input/pretrain-results/results-ensembled/results-
ensembled/bagging_results.csv'
best_f1_csv_path = '/kaggle/working/best_f1_summary.csv'

# Load data
bag_df = pd.read_csv(bag_path)

# Step 1: Best F1-score per CNN
best_bag_per_cnn = bag_df.groupby('CNN_Model')['F1_score'].max()

# Bar plot
plt.figure(figsize=(10, 5))
```

```
best_bag_per_cnn.plot(kind='bar', color='lightcoral')
plt.ylabel('Best F1-score (Bagging)')
plt.title('Best F1-score of Bagging (Linear SVM) for Each CNN
Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Step 2: Best overall result
best_bag_row = bag_df.loc[bag_df['F1_score'].idxmax()]
print(f"[Bagging] Best CNN: {best_bag_row['CNN_Model']}, F1 =
{best_bag_row['F1_score']:.4f}, "
      f"Params: {best_bag_row['Param_1']}")
"""
# Step 3: Append to summary CSV
pd.DataFrame([{
    'Method': 'Bagging (Linear SVM)',
    'CNN_Model': best_bag_row['CNN_Model'],
    'F1_score': best_bag_row['F1_score']
}]).to_csv(best_f1_csv_path, mode='a', header=not
os.path.exists(best_f1_csv_path), index=False)
"""

plt.figure(figsize=(12, 6))

# Plot each CNN model's curve
for model_name in bag_df['CNN_Model'].unique():
    subset = bag_df[bag_df['CNN_Model'] == model_name]
    n_vals = subset['Param_1'].str.extract(r'n_estimators=(\
d+)').astype(int)[0]
    plt.plot(n_vals, subset['F1_score'], marker='o', label=model_name)

plt.title("Bagging: F1-score vs Number of Estimators (All CNN
Models)")
plt.xlabel("Number of Estimators")
plt.ylabel("F1-score")
plt.ylim(0, 1)
plt.grid(True)
plt.legend(title="CNN Model")
plt.tight_layout()
plt.show()
```




```
[Bagging] Best CNN: InceptionV3, F1 = 0.8533, Params: n_estimators=20
```



Observations from Bagging (Linear SVM) Results

Best overall CNN extractor:
InceptionV3 achieved the highest F1-score (~0.853) using **20 bagging estimators**, highlighting its strong feature representations and synergy with ensemble methods.

- **MobileNetV2** followed closely behind InceptionV3, also delivering high F1-scores, confirming its robustness across ensemble-based setups.
- **ResNet50** showed solid performance, ranking just below the top two models.

- **VGG16** had moderate results, outperforming EfficientNetB0 but lagging behind the others.
- **EfficientNetB0** achieved the lowest F1-scores among all tested CNN feature extractors, indicating reduced effectiveness when used with Bagging and linear SVM.

From the estimator comparison plots (F1-score vs Number of Estimators):

- **Performance improved with more estimators:**
  Most CNNs showed slightly better or stable F1-scores as the number of estimators increased from 10 to 20, suggesting that additional ensemble members can help improve generalization.
- **InceptionV3 and MobileNetV2 showed the most consistent gains:**
  These models displayed robust and steady performance increases as more estimators were added, reinforcing their suitability for ensemble learning.
- **Efficiency trade-offs:**
  Although models like EfficientNetB0 had lower absolute F1-scores, they remained stable across different estimator counts, implying less benefit from additional ensemble complexity.

These results confirm that **InceptionV3** and **MobileNetV2** are highly compatible with bagging ensemble strategies, especially when combined with a sufficient number of base estimators and linear decision boundaries.

AdaBoost on Extracted CNN Features

We train an **AdaBoost ensemble model** using a shallow `DecisionTreeClassifier` (depth=1) as the base estimator, applied to feature vectors extracted from various pretrained CNN models.

What We Do:

- Define a grid search over:
  - **Number of estimators** (`n_estimators`): 100, 150, 200
- For a selected CNN feature extractor:
  - Extract feature vectors from the dataset
  - Split data into training and test sets
  - Train an AdaBoost classifier with a decision tree base for each `n_estimators` configuration
  - Evaluate each configuration using F1-score
- Save the results into a CSV file for downstream analysis

What We Visualize:

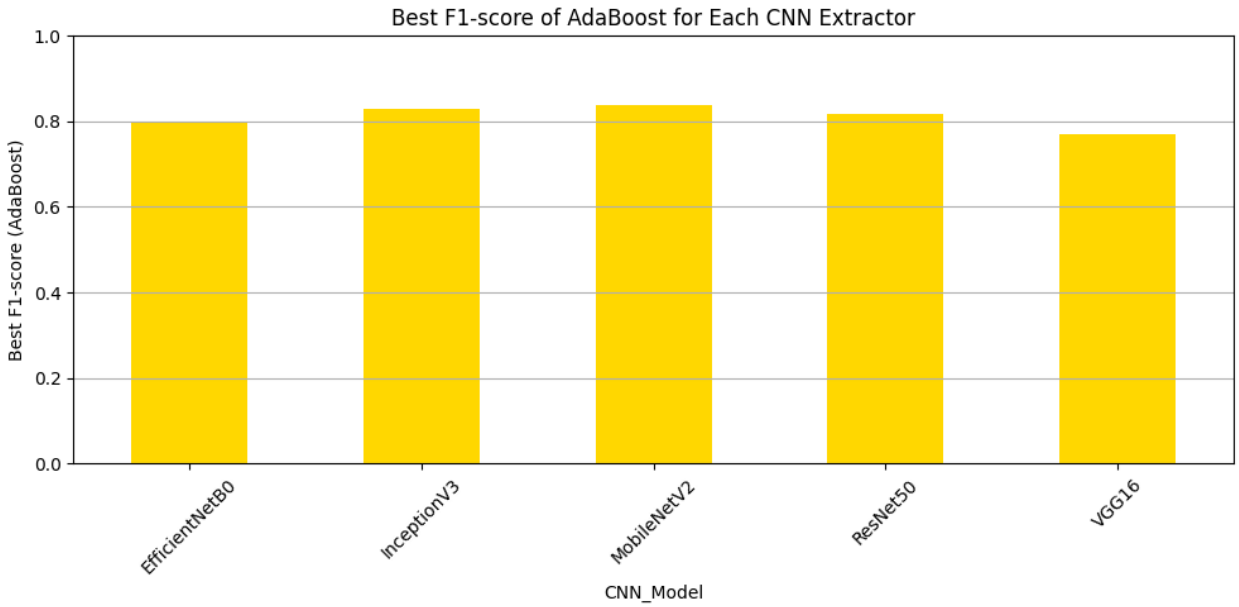
- **Best F1-score per CNN**: allows us to identify which CNN extractor performs best when paired with AdaBoost
- **F1-score vs Number of Estimators** for each CNN, highlighting how ensemble size affects classification performance

---

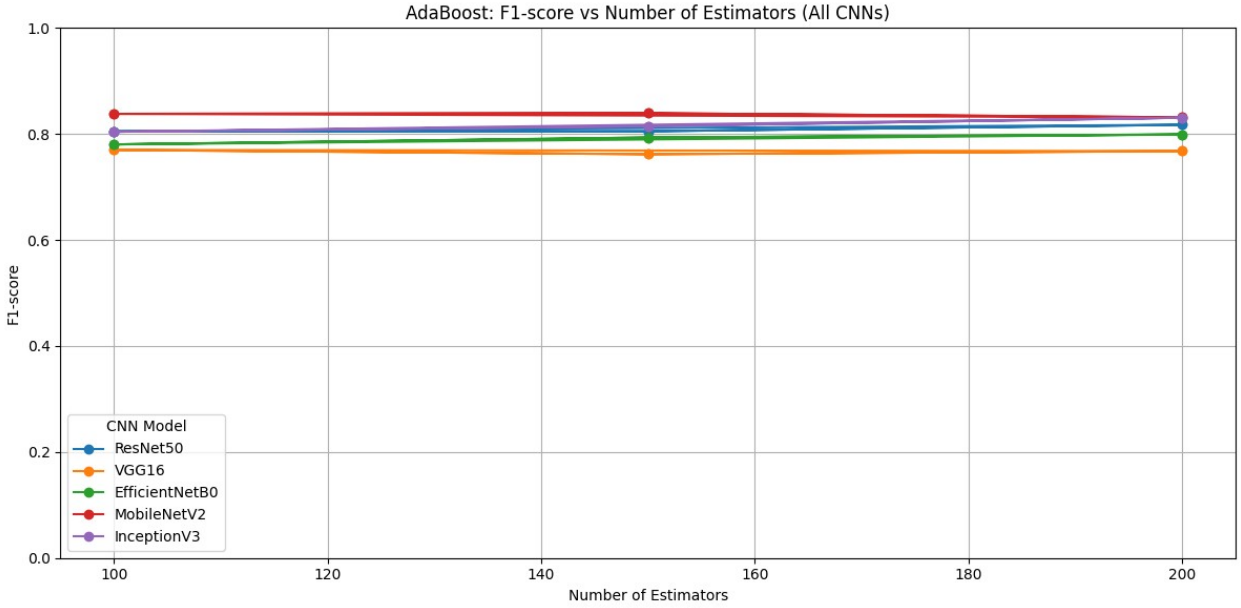
Note: AdaBoost training was executed separately for each CNN (i.e., `cnn_names[0]` to `cnn_names[4]`) because running all configurations in a single session exceeded GPU memory, leading to session crashes. This approach ensured training stability and consistent results across all configurations.

```
adaboost_n_estimators = [100, 150, 200]

model_name = cnn_names[0]
print(f"\n=== AdaBoost on CNN: {model_name} ===")
adaboost_results = []

features = extract_features(model_name, images)
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.2, stratify=labels, random_state=42
)

for n in adaboost_n_estimators:
    print(f"[AdaBoost] CNN={model_name} | n_estimators={n} |
base=DecisionTree(max_depth=1)")
    base_tree = DecisionTreeClassifier(max_depth=1, random_state=42)
    model = AdaBoostClassifier(base_estimator=base_tree,
n_estimators=n, random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred)
    adaboost_results.append({
        'CNN_Model': model_name,
        'Method': 'AdaBoost',
        'Param_1': f'n_estimators={n}',
        'Param_2': 'base=DecisionTree',
        'F1_score': f1
    })
    del model
    gc.collect()

df = pd.DataFrame(adaboost_results)
df.to_csv('/kaggle/working/adaboost_results.csv', mode='a', header=not
os.path.exists('/kaggle/working/adaboost_results.csv'), index=False)

# Paths
ada_path = '/kaggle/input/pretrain-results/results-ensembled/results-
ensembled/adaboost_results.csv'
```

```
best_f1_csv_path = '/kaggle/working/best_f1_summary.csv'

# Load data
ada_df = pd.read_csv(ada_path)

# Step 1: Best F1-score per CNN
best_ada_per_cnn = ada_df.groupby('CNN_Model')['F1_score'].max()

# Bar plot
plt.figure(figsize=(10, 5))
best_ada_per_cnn.plot(kind='bar', color='gold')
plt.ylabel('Best F1-score (AdaBoost)')
plt.title('Best F1-score of AdaBoost for Each CNN Extractor')
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis='y')
plt.tight_layout()
plt.show()

# Step 2: Best overall result
best_ada_row = ada_df.loc[ada_df['F1_score'].idxmax()]
print(f"[AdaBoost] Best CNN: {best_ada_row['CNN_Model']}, F1 =
{best_ada_row['F1_score']:.4f}, "
      f"Params: {best_ada_row['Param_1']}")
"""
# Step 3: Append to summary CSV
pd.DataFrame([{
    'Method': 'AdaBoost',
    'CNN_Model': best_ada_row['CNN_Model'],
    'F1_score': best_ada_row['F1_score']
}]).to_csv(best_f1_csv_path, mode='a', header=not
os.path.exists(best_f1_csv_path), index=False)
"""
plt.figure(figsize=(12, 6))

# Loop through each CNN model and plot its curve
for model_name in ada_df['CNN_Model'].unique():
    subset = ada_df[ada_df['CNN_Model'] == model_name]
    n_vals = subset['Param_1'].str.extract(r'n_estimators=(\
d+)').astype(int)[0]
    plt.plot(n_vals, subset['F1_score'], marker='o', label=model_name)

plt.title("AdaBoost: F1-score vs Number of Estimators (All CNNs)")
plt.xlabel("Number of Estimators")
plt.ylabel("F1-score")
plt.ylim(0, 1)
plt.grid(True)
plt.legend(title="CNN Model")
plt.tight_layout()
plt.show()
```




```
[AdaBoost] Best CNN: MobileNetV2, F1 = 0.8392, Params:
n_estimators=150
```



Observations from AdaBoost Results

Best overall CNN extractor:
MobileNetV2 achieved the highest F1-score (~0.839) using **150 AdaBoost estimators**, confirming its consistency as a top performer across ensemble strategies.

- **InceptionV3** closely followed, exhibiting stable and strong F1-scores, which supports its robust feature representations.

- **ResNet50** performed comparably to the top models, maintaining solid F1-scores across all tested estimator counts.
- **VGG16** showed the lowest F1-score among the models, consistent with previous observations across methods.
- **EfficientNetB0** remained in the lower range of performance, though it demonstrated slight gains with increased estimators.

From the estimator comparison plots (F1-score vs Number of Estimators):

- **More estimators improved performance:**
  Most CNN-based AdaBoost models saw improved or stable F1-scores as the number of estimators increased from 100 to 200.
- **MobileNetV2 and InceptionV3 again showed consistent performance:**
  These models maintained strong, stable curves, indicating they benefit most from boosting with shallow decision stumps.
- **Diminishing returns for some models:**
  For VGG16 and EfficientNetB0, increasing the number of estimators did not yield significant performance improvements, suggesting limited compatibility with boosting strategies.

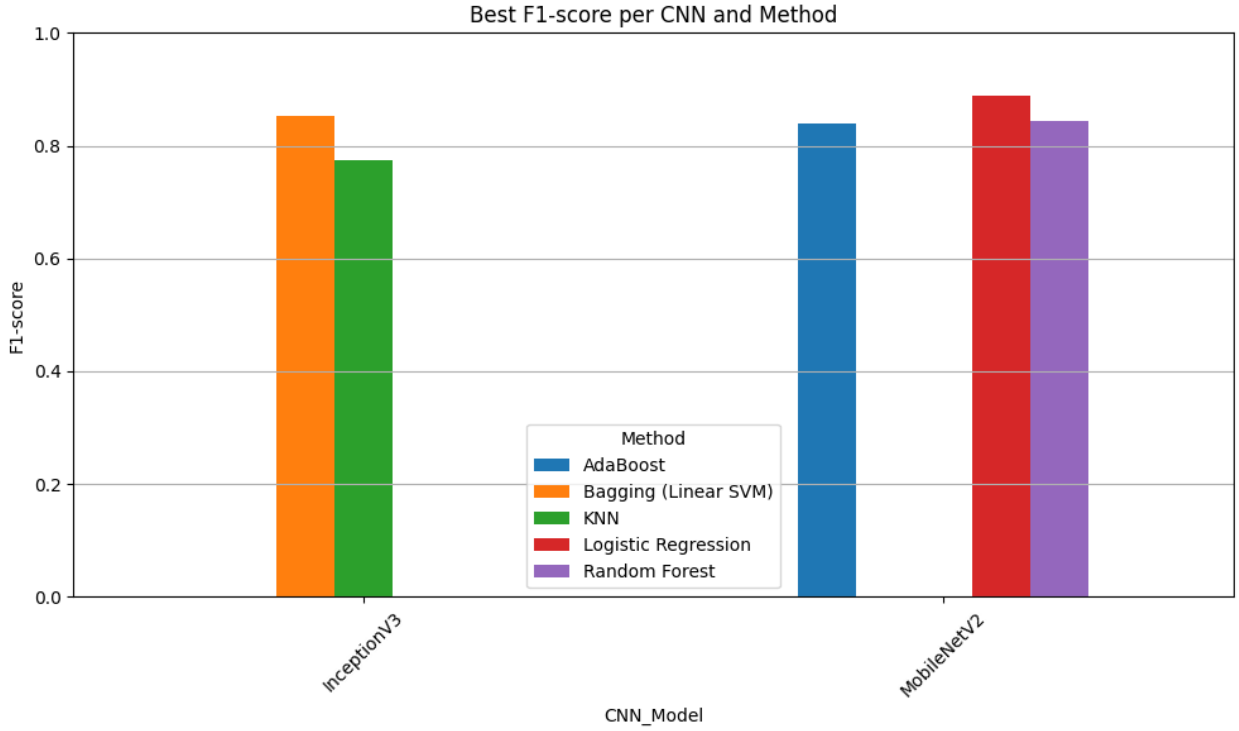
These findings reinforce that **MobileNetV2** and **InceptionV3** are ideal candidates for ensemble methods like AdaBoost, offering reliable improvements with larger estimator counts and shallow base learners.

Final Evaluation: Comparing Best F1-Scores Across Methods

This bar plot displays the highest F1-score achieved by each CNN feature extractor across different classification methods.
Each bar represents the best-performing configuration for a specific CNN-method combination, allowing a clear visual comparison of which models and techniques worked best together.

```
# Read best F1-scores summary

# Path to store the best F1-score summary
#best_f1_csv_path = '/kaggle/working/best_f1_summary.csv'
best_f1_csv_path =
'/kaggle/input/pretrain-results/results/results/best_f1_summary.csv'
summary_df = pd.read_csv(best_f1_csv_path)


# Pivot table to get Method as columns, CNN_Model as index
```

```
pivot_df = summary_df.pivot_table(index='CNN_Model', columns='Method',
values='F1_score', aggfunc='max')

# Fill missing values (if any)
pivot_df = pivot_df.fillna(0)

# Plot
pivot_df.plot(kind='bar', figsize=(10, 6))
plt.title("Best F1-score per CNN and Method")
plt.ylabel("F1-score")
plt.ylim(0, 1)
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.legend(title='Method')
plt.tight_layout()
plt.show()
```



Observations from Best F1-Score Comparison

- **Top-performing CNNs**:
  MobileNetV2 and InceptionV3 emerged as the most consistently effective feature extractors across all evaluated methods.

- **MobileNetV2**:
  Achieved the highest F1-score overall with **Logistic Regression**, followed closely by **Random Forest** and **AdaBoost**.

This highlights its strong generalization capabilities, especially when paired with both simple and ensemble-based classifiers.

- **InceptionV3**:
  Performed best with **Bagging (Linear SVM)** and **KNN**, showing a synergy with models that benefit from high-dimensional representations and distance-based measures.

- **Method performance trends**:
  - **Logistic Regression** was surprisingly competitive, especially with MobileNetV2, showcasing that linear models can be effective when combined with strong feature extractors.
  - **Bagging** and **Random Forest** delivered robust results on both CNNs, confirming the benefit of ensemble methods.
  - **KNN** was the weakest performer among the methods, though it still showed solid results with InceptionV3.
- **Consistency across models**:
  MobileNetV2 displayed stable and high F1-scores across all classifiers, reinforcing its adaptability and reliability for downstream tasks.

These findings suggest that **MobileNetV2** is the most versatile and high-performing feature extractor, particularly when used with lightweight or ensemble classifiers.
InceptionV3 is also a strong candidate, especially in scenarios favoring distance-based or ensemble learning strategies.

4. Predict in Test Set using our CNN

This step loads and preprocesses the test images to prepare them for inference using the best CNN model from Section 2 (with optimized hyperparameters).

What We Do:

- Load test image filenames and IDs.
- Read and resize each image to 224×224.
- Normalize pixel values to [0, 1].
- Store all images in `X_test` for model prediction.

```
# Load the test image filenames
test_ids_df = pd.read_csv(test_csv_path)
test_filenames = test_ids_df['Filename'].tolist()
test_ids = test_ids_df['ID'].tolist()


# Load and preprocess test images using your resize_images function
def load_and_preprocess_test_images(test_images_dir, filenames,
```

```
resize_func, target_dim=(224, 224)):
    test_images = []
    loaded_filenames = []

    for filename in filenames:
        img_path = os.path.join(test_images_dir, filename)
        img = cv2.imread(img_path)
        if img is not None:
            test_images.append(img)
            loaded_filenames.append(filename)

    print(f"Loaded {len(test_images)} test images.")

    resized_images, scaling_info = resize_func(test_images,
target_dim=target_dim)
    resized_images = np.array(resized_images).astype('float32') /
255.0

    return resized_images, loaded_filenames, scaling_info

# Apply the loading and preprocessing
test_images, ordered_filenames, _ = load_and_preprocess_test_images(
    test_images_dir,
    test_filenames,
    resize_images,
    target_dim=(224, 224)
)

X_test = test_images
```

```
Loaded 500 test images.
```

Final CNN Training with Best Hyperparameters

This code trains a CNN using the best hyperparameters found from Section 2. A `StopOnLoss` callback is implemented to stop training early once a desired loss threshold is reached.

What We Do:

- Extract best configuration from search results.
- Build a CNN with the optimal number of convolutional and dense layers, filters, units, and dropout rate.
- Train the model with early stopping based on validation loss.
- Plot training history (accuracy and loss).
- Evaluate the final model on the validation set.

```
class StopOnValLoss(Callback):
    def __init__(self, threshold=0.195):
```

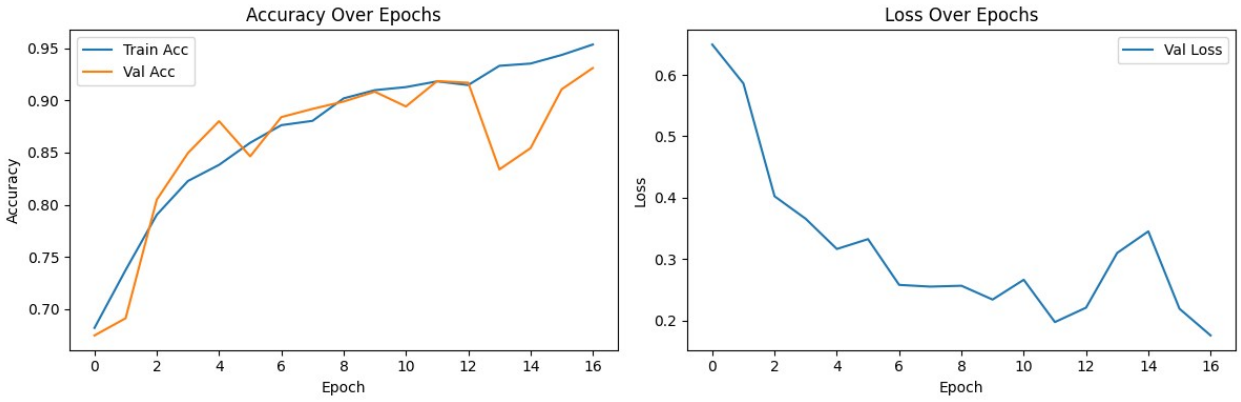
```
        super().__init__()
        self.threshold = threshold

    def on_epoch_end(self, epoch, logs=None):
        val_loss = logs.get('val_loss')
        if val_loss is not None and val_loss < self.threshold:
            print(f"\nStopping early: val_loss has reached
{val_loss:.4f} < {self.threshold}")
            self.model.stop_training = True


#early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
early_val_loss = StopOnValLoss(threshold=0.19)

# Step 2: Unpack parameters
num_conv_layers = int(best_config['num_conv_layers'])
num_dense_layers = int(best_config['num_dense_layers'])
conv_filters = int(best_config['conv_filters'])
dropout_rate = float(best_config['dropout_rate'])
batch_size = int(best_config['batch_size'])
dense_units = int(best_config['dense_units'])


# Step 3: Rebuild and train the best model
input_shape = X_train.shape[1:]  # (height, width, channels)

model = Sequential()

# First Conv layer
model.add(Conv2D(conv_filters, (3, 3), activation='relu',
input_shape=input_shape))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))

# Additional Conv layers
for i in range(1, num_conv_layers):
    filters = min(conv_filters * (2 ** i), 128)  # cap filters
    model.add(Conv2D(filters, (3, 3), activation='relu'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2, 2)))

model.add(Flatten())

# Dense layers with progressive reduction
units = 256
model.add(Dense(units, activation='relu'))
model.add(Dropout(0.5))

units = 128
```

```
model.add(Dense(units, activation='relu'))
model.add(Dropout(0.3))

# Output layer
model.add(Dense(2, activation='softmax'))

model.compile(optimizer=Adam(),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Step 4: Train longer
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=20,  # Increase if needed
    batch_size=batch_size,
    callbacks=[early_val_loss],
    verbose=1
)

# Step 5: Plot training history
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Accuracy Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()

# Step 6: Evaluate final model
results = evaluate_model(model, X_val, y_val, plots=True)
```

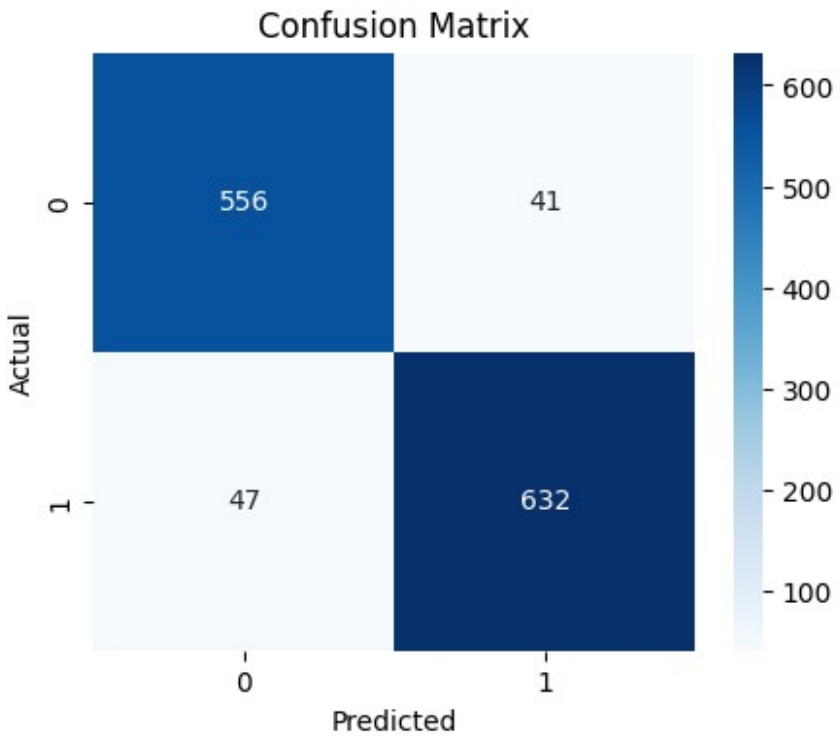
```
I0000 00:00:1746702141.990700      31 gpu_device.cc:2022] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 15513 MB
memory:  -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id:
0000:00:04.0, compute capability: 6.0

Epoch 1/20
```

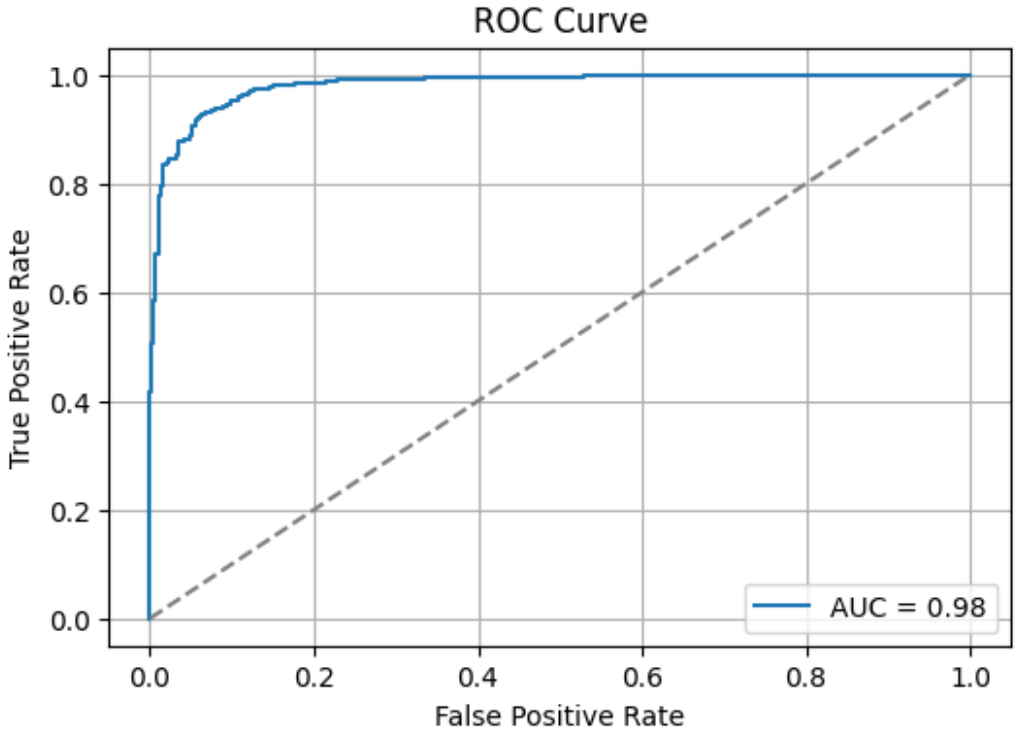
```
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
I0000 00:00:1746702156.612910     176 service.cc:148] XLA service
0x7abcd4003a60 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
I0000 00:00:1746702156.613965     176 service.cc:156]   StreamExecutor
device (0): Tesla P100-PCIE-16GB, Compute Capability 6.0
I0000 00:00:1746702157.135405     176 cuda_dnn.cc:529] Loaded cuDNN
version 90300

  7/160 ━━━━━━━━━━━━━━━━━━━━ 3s 21ms/step - accuracy: 0.5308 - loss:
3.8900

I0000 00:00:1746702162.065277     176 device_compiler.h:188] Compiled
cluster using XLA!  This line is logged at most once for the lifetime
of the process.

160/160 ━━━━━━━━━━━━━━━━━━━━ 22s 74ms/step - accuracy: 0.6610 - loss:
2.6356 - val_accuracy: 0.6748 - val_loss: 0.6495
Epoch 2/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.7438 - loss:
0.9154 - val_accuracy: 0.6912 - val_loss: 0.5861
Epoch 3/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.7731 - loss:
0.5373 - val_accuracy: 0.8049 - val_loss: 0.4023
Epoch 4/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.8184 - loss:
0.4131 - val_accuracy: 0.8495 - val_loss: 0.3656
Epoch 5/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.8354 - loss:
0.3862 - val_accuracy: 0.8801 - val_loss: 0.3165
Epoch 6/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.8535 - loss:
0.3245 - val_accuracy: 0.8464 - val_loss: 0.3325
Epoch 7/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.8773 - loss:
0.3056 - val_accuracy: 0.8840 - val_loss: 0.2580
Epoch 8/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.8683 - loss:
0.3200 - val_accuracy: 0.8918 - val_loss: 0.2552
Epoch 9/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9065 - loss:
0.2248 - val_accuracy: 0.8989 - val_loss: 0.2566
Epoch 10/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9106 - loss:
0.2250 - val_accuracy: 0.9083 - val_loss: 0.2341
Epoch 11/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9075 - loss:
0.2287 - val_accuracy: 0.8942 - val_loss: 0.2663
Epoch 12/20
```

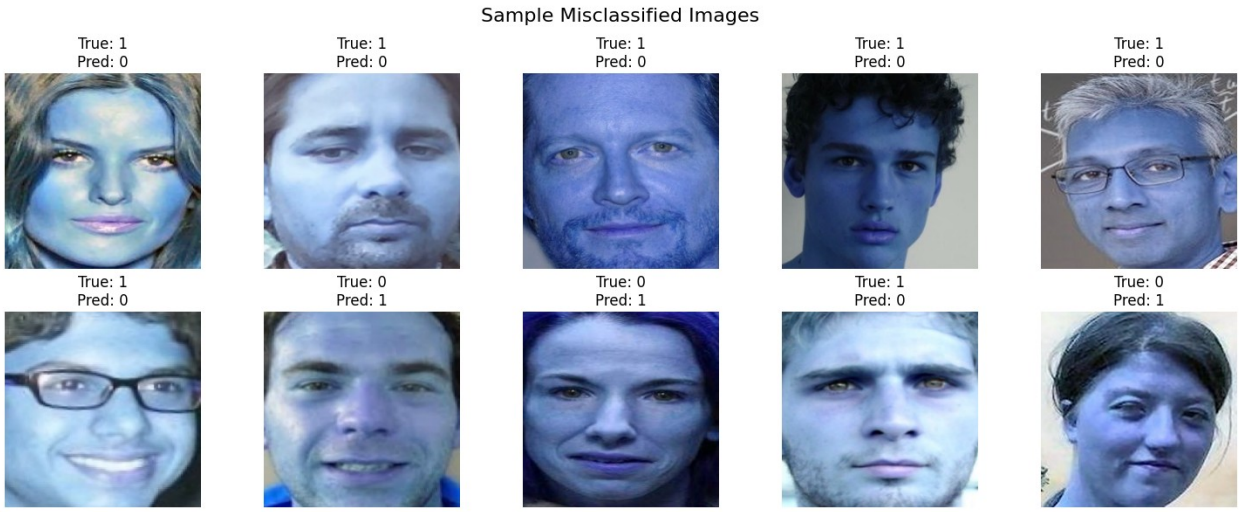
```
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9186 - loss:
0.1823 - val_accuracy: 0.9185 - val_loss: 0.1974
Epoch 13/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9168 - loss:
0.1881 - val_accuracy: 0.9169 - val_loss: 0.2208
Epoch 14/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9308 - loss:
0.1782 - val_accuracy: 0.8339 - val_loss: 0.3100
Epoch 15/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9310 - loss:
0.1609 - val_accuracy: 0.8542 - val_loss: 0.3452
Epoch 16/20
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9464 - loss:
0.1478 - val_accuracy: 0.9107 - val_loss: 0.2191
Epoch 17/20
157/160 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9534 - loss:
0.1231
Stopping early: val_loss has reached 0.1756 < 0.19
160/160 ━━━━━━━━━━━━━━━━━━━━ 4s 24ms/step - accuracy: 0.9534 - loss:
0.1231 - val_accuracy: 0.9310 - val_loss: 0.1756
```




```
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step
```


Confusion Matrix
556
41
47
632
Actual
Predicted
0
1
600
500
400
300
200
100


ROC Curve
True Positive Rate
False Positive Rate
AUC = 0.98
0.0
0.2
0.4
0.6
0.8
1.0




```
 - F1 Score:  0.9349
 - ROC AUC:   0.9819
```

```
# Predict on test data
y_prob = model.predict(X_test)
y_pred = np.argmax(y_prob, axis=1)

# Build submission DataFrame
submission_df = pd.DataFrame({
    'ID': test_ids,
    'LABEL': y_pred
})

# Sort by ID (optional but recommended)
submission_df = submission_df.sort_values('ID')

# Save to CSV
submission_df.to_csv('submission.csv', index=False)

# Show sample output
print(submission_df.head(100))
FileLink('submission.csv')
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 56ms/step
    ID  LABEL
0    0      0
1    1      1
2    2      0
3    3      1
4    4      1
..  ..    ...
95  95      0
96  96      1
97  97      1
```

```
98  98      0
99  99      1

[100 rows x 2 columns]

/kaggle/working/submission.csv
```